LakeCat

covers lakecat (0.1.0)

An Engine-Close Catalog Foundation for QueryGraph

Alexy Khrabrov

chiefscientist.org

&

Codex with ChatGPT 5.5

Contents

1 LakeCat

1.1 Preface

LakeCat is a Rust-native Iceberg catalog foundation for QueryGraph. It starts from a deliberately conservative claim: the ordinary Iceberg REST catalog boundary must keep working for ordinary engines, but the next catalog also needs to become a governed control plane for Rust-first planning, semantic graph handoff, lineage, and agent access.

In this repository the catalog is LakeCat and the engine-side lakehouse implementation is Sail. The idea is not to invent a new table format or to make every client learn a new protocol. It is to keep Iceberg compatibility at the boundary while moving the parts that need deep Iceberg knowledge closer to the Rust engine that can reason about them.

This book starts from first principles. It explains what a catalog is, why Apache Iceberg puts so much responsibility into metadata, what Sail already does with Iceberg metadata, and why a passive catalog is not enough for governed agentic systems. Then it shows the LakeCat shape: a thin Rust catalog that owns identity, tenancy, metadata-pointer state, policy gates, idempotent commits, and integration events, while delegating reusable engine, graph, and security work to Sail, Grust, and TypeSec.

The intended end state is QueryGraph.ai. LakeCat should become the catalog foundation for the next QueryGraph: a place where standard engines still see an Iceberg REST catalog, while QueryGraph can bootstrap Croissant, CDIF, OSI, ODRL, OpenLineage, TypeSec security receipts, and a Grust-backed graph from the same governed source of truth.

1.2 What A Catalog Is

A data catalog is often described as a place that lists datasets. That is true, but too small. A real catalog is the control plane between names, storage, metadata, identity, and intent.

At minimum, a catalog answers four questions:

1. What table does this name mean?
2. Where is its current metadata?
3. Who is allowed to read, write, plan, or administer it?
4. What changed, when, and under whose authority?

In a traditional database, the catalog is embedded inside the database system. The engine owns the table definitions, statistics, indexes, permissions, and transaction log. A client asks the database a question and the same system resolves names, checks permissions, plans the query, and executes it.

A lakehouse splits that system apart. Data files live in object storage. Metadata files live beside them. Multiple engines may read and write the same tables. A catalog becomes the agreement point: it maps a logical table name to the current metadata pointer and arbitrates updates to that pointer.

That pointer is deceptively important. If the catalog points at metadata version 17, the table is version 17. If a writer prepares version 18 and wins the compare-and-swap update, the table becomes version 18. If it loses, the table does not partially change. The catalog is not the table format, but it is the place where table history becomes visible and durable.

For human-scale analytics this can sound like bookkeeping. For agentic systems it becomes a trust boundary. A catalog can know the principal, the warehouse, the namespace, the table, the current snapshot, the requested columns, the row restriction, the storage profile, and the policy receipt. If that information is captured before planning and committed after state changes, the catalog becomes the control plane for governed data access rather than a passive address book.

1.3 What Iceberg Does

Apache Iceberg is a table format for large analytic tables. Its core design is simple: put the table's truth in explicit metadata files, and let engines use that metadata to plan reads and validate writes without relying on directory listing or fragile storage conventions.

An Iceberg table has a current metadata file. That metadata names schemas, partition specs, sort orders, snapshots, properties, and the current snapshot. Snapshots point to manifest lists. Manifest lists point to manifests. Manifests describe data files and delete files. The data files normally use formats such as Parquet, Avro, or ORC.

This layered metadata gives Iceberg its practical power:

- Schema evolution is explicit.
- Partition evolution is explicit.
- Snapshot isolation is explicit.
- Commit conflicts can be checked before the current pointer advances.
- Scan planning can prune manifests and files before touching data.
- Deletes can be represented without rewriting every data file immediately.
- Multiple engines can interoperate because the table state is stored in a shared, documented format.

The catalog role in Iceberg is intentionally narrow. Standard clients need to load table metadata, create namespaces and tables, commit changes, and sometimes receive credentials or scan tasks. The catalog must not require business semantics or proprietary metadata for normal table access. If standard clients have to call a non-standard endpoint to read an ordinary table, compatibility is already broken.

LakeCat preserves that rule. Iceberg metadata stays pristine. Business semantics, policy, graph, lineage, and agent state are derived control-plane or graph data. The table remains an Iceberg table.

1.4 Standard Terms And LakeCat Terms

LakeCat is easiest to understand when the words are separated into three layers: standard Iceberg vocabulary, LakeCat implementation choices, and QueryGraph/TypeSec control-plane vocabu-

lary. Mixing those layers is where catalog designs become confusing. LakeCat deliberately keeps the ordinary Iceberg words ordinary, then adds stronger evidence and governance around them.

The following concept map is the working contract. Each entry separates the standard Iceberg meaning from LakeCat's implementation and from the QueryGraph/TypeSec extension surface.

Catalog: In standard Iceberg, a catalog resolves table identifiers, namespaces, and current table metadata locations. It is the control point that lets engines load table metadata and make optimistic updates without agreeing on one execution engine. In LakeCat, the catalog is also a Rust service spine under `/catalog/v1` with management and QueryGraph surfaces beside it. That spine owns tenancy, request identity, durable catalog state, and replayable evidence. The Rust implementation is not an Iceberg extension. It is how LakeCat makes a standard catalog reliable enough to become QueryGraph's foundation.

Namespace: In standard Iceberg, a namespace groups tables and views. In LakeCat, a namespace is also a governed resource under a warehouse. Namespace creation, listing, and lifecycle events can be authorized, audited, replay-validated, and projected as graph anchors. The grouping is standard Iceberg parlance. The authorization receipt, outbox record, and QueryGraph graph anchor are optional control-plane evidence.

Table identifier: In standard Iceberg, a table identifier is the catalog-relative table name, usually namespace plus table name. In LakeCat, that same identity becomes the root key for store rows, audit events, outbox messages, TypeSec contexts, and QueryGraph handoff bundles. The name is standard. The durable evidence envelope around it is LakeCat.

Current metadata location: In standard Iceberg, the current metadata location is the pointer to the active Iceberg table metadata JSON file. In LakeCat, that pointer is the central compare-and-swap value in the store, with pointer-log history and redacted hash evidence. Pointer history is useful enough to be a future Iceberg REST management extension, but the table itself is still defined by the standard metadata pointer.

Table metadata: In standard Iceberg, table metadata is the JSON metadata file containing schemas, snapshots, partition specs, sort orders, properties, and related table state. LakeCat stores that file in object storage and keeps it pristine. It validates and references the metadata, but it does not use custom business fields to carry policy, graph, lineage, or agent state. QueryGraph derives semantic and governance facts beside the metadata, not inside it.

Snapshots, manifest lists, manifests, data files, and delete files: These are standard Iceberg metadata layers used by engines to plan reads and validate table state. LakeCat delegates their interpretation to Sail for scan planning, manifest expansion, pruning, delete-file handling, and metadata-as-data work. They are engine responsibilities. Pushing them into Sail avoids a second partial Iceberg engine inside the catalog.

Iceberg REST paths: The standard Iceberg REST catalog path is the compatibility boundary. LakeCat serves table and namespace operations under `/catalog/v1` so ordinary engines can create namespaces, load tables, commit metadata updates, and use compatible scan or credential flows without learning QueryGraph. Management paths such as `/management/v1` and bootstrap paths

such as `/querygraph/v1/bootstrap` are LakeCat/QueryGraph additions beside the standard surface. They must never become prerequisites for ordinary table access.

Commit: In standard Iceberg, a commit is an optimistic update that advances the current metadata pointer when requirements still hold. In LakeCat, that standard commit is wrapped in a catalog transaction: request normalization, TypeSec receipt capture, Sail validation, create-only metadata writes, compare-and-swap, idempotency records, audit, pointer logs, and outbox events. The optimistic commit is standard. The receipt, pointer-log, and outbox evidence are LakeCat extensions around the standard commit.

Compare-and-swap: Compare-and-swap is the catalog-side atomicity rule behind optimistic Iceberg commits. A writer can advance the pointer only if the current metadata location still matches the expected state and the update requirements remain valid. In LakeCat, CAS is hardened by the store contract and by audit-safe conflict evidence: failed races expose hashes and structured error classes rather than raw storage paths. CAS itself is standard catalog behavior. LakeCat's redacted proof envelope is an implementation and governance extension.

Idempotency: Idempotency is the retry discipline that prevents duplicate effects. LakeCat makes it a hardened store contract: exact replay returns the stored response, different bodies under the same key conflict, and replay cannot emit duplicate outbox events. Iceberg can benefit from stronger cross-catalog idempotency conventions, but LakeCat treats its concrete key rules as catalog implementation behavior today and as a possible future REST profile rather than a table-format change.

Pointer log: A pointer log is LakeCat's compact history of accepted metadata-pointer movement. Iceberg tables already have snapshot and metadata history, but a catalog pointer log answers a different operational question: which catalog transaction advanced which pointer under which principal, request hash, response hash, policy hash, and idempotency key hash? This is not standard Iceberg today. It is an optional management surface and a strong candidate for future Iceberg REST or OpenLineage-adjacent catalog event conventions.

Audit: Audit is not an Iceberg table-format concept. LakeCat writes audit records for governed catalog actions so operators can reconstruct who did what, which authority was used, and which redacted evidence was captured. Audit belongs to the catalog control plane. It should stay outside Iceberg metadata, because a portable table should not be forced to carry one deployment's governance log.

Outbox: The outbox is LakeCat's transactional delivery buffer for committed catalog facts. LakeCat writes outbox events with catalog transactions, validates replay evidence before delivery, projects to Grust and OpenLineage, and acknowledges only after projection succeeds. The pattern is not standard Iceberg, but it is one of LakeCat's most important catalog extensions. It is also one of the best places to propose future interoperability: event identity, replay ordering, lineage binding, and redaction shape could become common catalog-event language without changing table metadata.

Replay validation: Replay validation is LakeCat's rule that durable evidence must be internally consistent before it can be acknowledged, projected to graph, or emitted as lineage. For example,

governed scan events must preserve matching `read-restriction` evidence in the top-level payload and the TypeSec receipt, and commit events must carry full request, response, idempotency, principal, and optional policy hashes. This is LakeCat control-plane hardening. The future standardization question is not whether Iceberg should require LakeCat's exact validators, but whether catalogs should share proof-carrying event profiles.

Credential vending: Iceberg REST catalogs may provide storage credentials or access material for table operations. LakeCat makes that path fail closed. Raw credentials are an audited exception; governed Sail-planned reads are the default for agents and untrusted principals. TypeSec receipts and credential-root proofs are LakeCat/TypeSec extensions. They are candidates for future governed-access conventions, not current Iceberg requirements.

Governed scan: Standard engines use Iceberg metadata to produce file tasks, apply projection, prune manifests and files, and account for deletes. LakeCat asks TypeSec for restrictions, passes effective projection, mandatory filters, purpose, and TTL constraints to Sail, records receipt evidence, and returns compatible plan/task shapes. Governed planning evidence is a LakeCat/TypeSec extension. The underlying pruning and delete semantics belong in Sail and the Iceberg engine layer.

Management surfaces: LakeCat management APIs expose warehouses, storage profiles, policy bindings, commit logs, view receipt chains, credential-root posture, and operational state. Iceberg REST does not standardize all of those surfaces today. Some are implementation administration APIs. Some are QueryGraph handoff APIs. Some, especially commit-history, credential, and event replay profiles, may be worth proposing as optional Iceberg REST management extensions.

View proof: Iceberg has view concepts and REST view endpoints, but LakeCat adds proof surfaces around view lifecycle, list evidence, and receipt chains so QueryGraph can verify that a view import corresponds to governed catalog state. The view itself should remain standard where the standard applies. The receipt chain is LakeCat/QueryGraph evidence beside it.

OpenLineage projection: OpenLineage is not Iceberg, but it is a natural consumer of catalog events. LakeCat projects committed namespace, table, scan, credential, view, and management events into OpenLineage from the durable outbox. That makes lineage reflect committed catalog state instead of handler-side best effort. This is an extension around Iceberg and a likely interoperability point, not a replacement for Iceberg metadata.

Graph projection: Graph projection is not an Iceberg table-format feature. LakeCat emits catalog-facing graph facts only at the boundary; graph taxonomy, storage, traversal, and query behavior live in Grust. QueryGraph builds the semantic graph from these anchors. The graph is an extension around Iceberg, not an alternative to Iceberg.

Policy receipt: Policy receipts are not an Iceberg table-format feature. LakeCat persists TypeSec-style authorization receipts and checks replay evidence before admitting outbox delivery. This belongs to TypeSec and governed catalog protocols. It may inspire future Iceberg governance extensions, but it should not be mandatory for ordinary table access.

Bootstrap bundle: A QueryGraph bootstrap bundle is not part of standard Iceberg. It is a handoff contract that packages catalog, table, namespace, view, lineage, management, credential, and commit proof surfaces. This is QueryGraph-specific integration. Standard clients should never need it.

Rust service spine: LakeCat's Rust service/catalog spine is an implementation choice with architectural consequences. It lets one process own request identity, Iceberg-compatible routing, Turso-backed catalog state, Sail calls, TypeSec receipts, and outbox projection without turning every boundary into a remote adapter. This is not a proposed Iceberg feature. The feature that might matter to Iceberg is the behavior it enables: deterministic commits, replayable catalog events, and engine-close governed planning.

Turso-backed local store: The Turso-backed store is LakeCat's durable local spine. It persists projects, warehouses, namespaces, tables, storage profiles, pointer logs, idempotency records, audit rows, and outbox rows through the Rust `turso` crate. This is not Iceberg parlance and not an Iceberg extension. It is LakeCat's chosen local implementation of the catalog-state contract, kept behind `CatalogStore` so the higher-level catalog behavior remains portable.

This separation answers an important design question: are LakeCat's additions Iceberg extensions, future Iceberg features, or something else?

Some additions are not extensions at all. The Rust service spine, Turso-backed local store, normalized idempotency table, pointer log, and replay validators are implementation choices behind standard catalog behavior. A Spark, Trino, Flink, or PyIceberg client does not need to know that LakeCat uses Rust, Turso, or a particular hash discipline to make a commit safe. Those details matter because they make the catalog reliable, portable, and inspectable, but they do not change what an Iceberg table is.

Some additions are catalog extensions. QueryGraph bootstrap, management replay, credential-root proofs, view proof chains, commit-history inspection, and OpenLineage projection are useful APIs beside the Iceberg REST surface. They should remain optional. A standard Iceberg client should be able to ignore them and still create, load, update, and read tables through the normal catalog paths.

Some additions are future Iceberg-adjacent candidates. The community may eventually want common language for idempotent commit replay, governed credential vending, catalog event streams, lineage receipts, policy-bound scan planning, or proof-carrying table/view management operations. LakeCat should be a good laboratory for those ideas, but it should not force them into Iceberg metadata or make them prerequisites for ordinary compatibility. The right shape is additive: standard REST stays stable, advanced governance evidence is discoverable, and engines that do not understand it keep working.

The clean line is this: implementation details make LakeCat reliable, optional extensions make QueryGraph rich, and future proposals should be phrased as additive catalog profiles rather than mandatory table-format changes. Iceberg metadata remains the shared table truth. LakeCat's proof surfaces explain how a catalog transaction, scan plan, credential decision, or semantic handoff happened around that table truth.

That is also why LakeCat is careful with the phrase "Iceberg v4 compatible." For the catalog, compatibility means preserving the standard contract while being ready for newer format metadata and REST models as Sail exposes typed support. It does not mean guessing future semantics in LakeCat or stuffing custom control-plane state into table metadata. JSON passthrough can keep the catalog tolerant during transition, but typed behavior should move into Sail as soon as Sail has the reusable model.

1.5 What Exists Today And What It Means

LakeCat is not only a design sketch. The current implementation already has a Rust service/catalog spine, a Turso-backed local store direction, Iceberg REST-compatible namespace and table paths, hardened commit and replay evidence, governed scan and credential proof, and broad QueryGraph/QGLake handoff surfaces. Those pieces are easy to over-describe as "Iceberg extensions," but that phrase hides the most important distinction: some pieces are ordinary catalog implementation, some are optional LakeCat/QueryGraph APIs, some are TypeSec governance proof, and only some are plausible future standardization candidates.

The Rust service/catalog spine exists today. Its job is to receive standard catalog traffic, normalize request identity, bind that request to warehouse and table state, call the local store, call Sail, call TypeSec, and write durable audit/outbox evidence. None of that is standard Iceberg vocabulary. Iceberg does not care whether a catalog is implemented in Rust, Java, Go, or as a managed cloud service. It cares that clients can create namespaces, load table metadata, commit compatible updates, and receive compatible responses. The Rust spine is therefore an implementation choice, not an Iceberg extension. It matters because it makes LakeCat fast enough and direct enough to be a QueryGraph foundation: request identity, commit CAS, policy receipts, Sail planning, and outbox projection can be kept in one strongly typed transaction path instead of being scattered across adapters.

The Turso-backed local store direction is in place. Turso is not part of Iceberg parlance either. It is LakeCat's durable local catalog database behind the portable `CatalogStore` trait. The standard Iceberg concept is the current metadata location and the optimistic update of that location. LakeCat's Turso store persists the things a catalog must remember around that pointer: warehouses, namespaces, table records, storage profiles, idempotency rows, pointer logs, audit rows, outbox rows, view records, policy bindings, and soft-delete state. The future-facing idea is not "Iceberg should use Turso." The future-facing idea is that Iceberg REST catalogs may benefit from clearer profiles for durable idempotency, pointer-history inspection, and event replay. Turso is LakeCat's Rust-local implementation of those behaviors.

Iceberg REST-compatible table and namespace paths exist. This is the standard surface. A normal client should be able to reach the catalog through `/catalog/v1`, create and list namespaces, load tables, commit table changes, and use compatible credential or scan flows without understanding QueryGraph. That compatibility boundary is non-negotiable. LakeCat-specific paths such as management APIs, QueryGraph bootstrap, QGLake handoff, replay verification, and proof inspection live beside the standard surface. They are optional control plane APIs. They can be useful to QueryGraph, operators, agents, and lineage systems, but they must not become hidden requirements for ordinary table access.

Commit CAS, idempotency, pointer logs, audit/outbox, and replay validation are heavily hardened in LakeCat. The standard Iceberg idea is simple and powerful: a commit advances the current metadata pointer only if requirements still hold. LakeCat keeps that idea, then surrounds it with production catalog discipline. CAS is performed by the store. Idempotency rejects different bodies under the same key and replays exact stored responses without duplicating side effects. Pointer logs preserve accepted pointer movement. Audit records preserve who and what authorized the operation. The outbox keeps graph and OpenLineage delivery transactional with catalog state. Replay validation checks that durable event evidence still agrees with the receipt, table identity, hash shape, counts, policy restrictions, credential posture, and principal before any projection is acknowledged. In Iceberg terms, only the optimistic pointer update is standard. The surrounding evidence system is LakeCat's catalog hardening. The standard candidate is an optional catalog-event or management profile, not a change to Iceberg metadata files.

Governed scan and credential paths carry substantial TypeSec-style receipt evidence. Standard Iceberg metadata gives engines enough information to plan files, prune manifests, and handle deletes. Standard REST catalogs can also vend credentials. LakeCat adds a governed path for principals, agents, and policy-bearing requests: it asks TypeSec for a decision, turns that decision into an effective read restriction, passes the narrowed request to Sail, records the receipt, and validates replay before graph or OpenLineage projection. Credential vending follows the same posture. Raw credentials are a trusted, audited exception; untrusted agents should receive Sail-planned work instead of broad storage power. This is not standard Iceberg today. It is a LakeCat/TypeSec governance extension and a strong candidate for future governed-access conventions. If standardized, it should remain additive: clients that understand the proof can verify it, and clients that do not can still use the standard catalog path.

QueryGraph and QGLake handoff surfaces are broad. LakeCat can expose bootstrap, management, view, credential, commit-history, OpenLineage, and replay proof material so QueryGraph can import a governed catalog state rather than scrape a database or infer semantics from object paths. That handoff is intentionally not Iceberg itself. QueryGraph owns Croissant, CDIF, OSI, ODRL interpretation at the semantic application layer, graph import semantics, agent workflow meaning, and higher-level business vocabulary. LakeCat supplies the governed catalog facts and proof anchors: which namespace, which table, which current pointer, which commit sequence, which credential decision, which view receipt, which scan restriction, which lineage hashes. QueryGraph decides what those facts mean in a semantic application.

The clean classification is:

- Standard Iceberg parlance: catalog, namespace, table identifier, current metadata location, table metadata, snapshots, manifests, data files, delete files, optimistic commit requirements, and REST catalog compatibility.
- LakeCat implementation: Rust service spine, Turso local store, internal store traits, request normalization, durable idempotency, redaction rules, pointer-log rows, audit rows, outbox rows, replay validators, and local release gates.

- LakeCat optional catalog extensions: management APIs, pointer-history reads, view receipt chains, storage-profile management, replay verification, QueryGraph bootstrap generation, QGLake compact handoff, and OpenLineage projection.
- TypeSec governance extensions: authorization receipts, capability decisions, TypeDID/agent identity context, ODRL-derived read restrictions, credential-root proof, governed-read evidence, and raw-credential exception receipts.
- QueryGraph application extensions: Croissant/CDIF/OSI/ODRL semantic import, graph-model bootstrap, agentic workflow proof, business semantic alignment, and QueryGraph import verification.
- Future Iceberg-adjacent candidates: optional profiles for idempotent commit replay, catalog event streams, governed credential vending, proof-carrying scan planning, pointer-history inspection, view lifecycle proof, and lineage receipt binding.

The last category should be handled carefully. LakeCat should not try to declare a private future Iceberg. It should implement useful behavior, keep it cleanly additive, publish precise proof shapes, and learn from real QueryGraph/QGLake workflows. Good future proposals come from proven interoperability pressure: several engines, catalogs, lineage systems, and governance systems discovering that they need the same optional evidence. Until then, the standard table remains standard, the extension remains optional, and the implementation remains replaceable.

The practical test is whether a normal Iceberg client can still succeed while ignoring the extra surface. If Spark loads a table, reads the current metadata location, and commits a compatible metadata update through `/catalog/v1`, it is using standard Iceberg catalog behavior. If QueryGraph later asks LakeCat for a bootstrap bundle containing graph anchors, OpenLineage receipts, view receipt chains, credential posture, and commit-history proof, it is using an optional LakeCat/QueryGraph extension. If an agent asks LakeCat for a governed plan and LakeCat asks TypeSec for a receipt before calling Sail, that is a governance extension around a standard table. None of those flows require LakeCat to add private fields to Iceberg metadata or require an ordinary client to understand QueryGraph semantics.

That distinction gives each concept a status:

1. **Required for Iceberg compatibility.** Namespaces, table identifiers, current metadata locations, table metadata, snapshots, manifests, delete files, optimistic commits, and REST-compatible routes must behave like Iceberg expects.
2. **Required for LakeCat reliability.** The Rust service spine, Turso store, CAS discipline, idempotency records, pointer logs, audit rows, outbox rows, replay validators, redaction rules, and local release gates make LakeCat a trustworthy implementation, but they are invisible to ordinary clients.
3. **Optional LakeCat/QueryGraph extensions.** Management routes, view proof, bootstrap bundles, QGLake compact handoff, OpenLineage projection, credential-root posture, and replay-verification endpoints enrich the catalog for operators and QueryGraph without changing table truth.

4. **Governance extensions owned with TypeSec.** Policy decisions, capability receipts, TypeDID context, ODRL-derived restrictions, secure-agent proof, and raw-credential exception evidence belong beside catalog operations, not inside table metadata.
5. **Possible future Iceberg proposals.** Only repeated interoperability need should turn LakeCat behavior into a proposal. Good candidates are optional catalog profiles for idempotent commit replay, pointer-history inspection, governed credential vending, catalog event streams, lineage receipt binding, and proof-carrying scan planning.

The word "extension" should therefore be used narrowly. Turso is not an Iceberg extension. Rust is not an Iceberg extension. A replay validator is not an Iceberg extension. Those are implementation details that make standard behavior stronger. QueryGraph bootstrap is an extension because it is a new surface beside Iceberg REST. Governed scan proof is an extension because it adds policy and receipt semantics around scan planning. A future Iceberg proposal should come only after those extensions prove useful enough that other catalogs and engines would benefit from a shared optional profile.

1.5.1 Standard Word, LakeCat Mechanism, Future Candidate

The distinction is easiest to keep honest by asking four questions for each concept:

1. Is this already Iceberg parlance?
2. Is this LakeCat's implementation of a standard catalog obligation?
3. Is this an optional LakeCat, QueryGraph, or TypeSec surface beside Iceberg?
4. Is this mature enough to become a future Iceberg proposal?

The Rust service spine answers the second question. Iceberg says a catalog must resolve identifiers, serve metadata, and commit compatible updates. It does not say the catalog must be written in a particular language. LakeCat chooses Rust so request identity, store transactions, Sail engine calls, TypeSec receipts, replay validation, and outbox projection can be expressed in one strongly typed service path. That is not an Iceberg extension. It is an implementation stance. The future Iceberg-relevant lesson is not "catalogs should be Rust," but "catalog APIs should make it possible to produce deterministic proof for commits, scans, credentials, and events."

The Turso-backed local store also answers the second question. Standard Iceberg needs atomic pointer movement and durable catalog state. LakeCat's Turso store is how the local Rust catalog persists that state without reintroducing a separate SQLx/SQLite abstraction. The interesting general idea is the store contract: CAS must be atomic, idempotency must reject drift, pointer logs must be replayable, audit and outbox rows must be transactionally tied to catalog state, and redaction must be stable. Turso is LakeCat's local durable spine. The possible future proposal is a catalog-state behavior profile, not a database choice.

Iceberg REST-compatible table and namespace paths answer the first question. They are standard catalog parlance. LakeCat's `/catalog/v1` surface must let ordinary clients create namespaces, list namespaces, load tables, commit table metadata, and interact with compatible scan or credential routes. LakeCat can add management, bootstrap, replay, and handoff APIs beside that surface, but those additions must never become prerequisites for a normal table read or commit. If a PySpark

or PyIceberg workflow succeeds by using only the standard catalog surface, compatibility is real. If it secretly depends on a QueryGraph-only route, compatibility has leaked.

Commit CAS is partly first-question and partly second-question. Optimistic catalog commits are central Iceberg behavior. LakeCat's CAS hardening is the implementation of that behavior under production pressure: object writes are create-only, pointer movement is atomic, conflict evidence is redacted, request and response hashes are preserved, and idempotency prevents duplicate side effects. The future Iceberg-adjacent candidate is a shared way to express idempotent commit replay and pointer-history proof. It should not change the table metadata format. It should describe how a catalog can prove a compatible commit happened once.

Pointer logs, audit, outbox, and replay validation answer the third question. They are not ordinary Iceberg table terms, but they are natural catalog-control terms. A pointer log records accepted pointer movement. Audit records preserve authority and request evidence. The outbox records durable facts that must be projected to graph and lineage. Replay validation proves that the durable fact still matches identity, table state, hashes, counts, policy restrictions, credential posture, and principal shape before delivery. These are LakeCat extensions around a standard catalog. They are also plausible future interoperability candidates because every serious catalog eventually has to answer the same operational question: how do I prove which catalog fact I emitted, under which authority, and from which committed state?

Governed scan and credential proof answer the third and fourth questions. The standard Iceberg world already has metadata, manifests, delete files, scan planning, and catalog credential vending. LakeCat adds TypeSec-style proof: which principal asked, which capability or TypeDID context applied, which ODRL or policy rule produced the restriction, which columns and rows survived, what purpose and TTL were allowed, and whether raw credentials were blocked or audited. That is not required for standard clients today. It is a governance extension. It may become a future Iceberg-adjacent profile only if the community wants a common governed-access shape that multiple catalogs and engines can verify.

QueryGraph and QGLake handoff answer the third question. They are intentionally application-facing. QueryGraph needs Croissant, CDIF, OSI, ODRL, OpenLineage, graph import, view proof, credential posture, management inventory, bootstrap artifacts, and commit-history proof. LakeCat should provide the catalog facts and evidence anchors. QueryGraph should compose the semantic graph and user workflow. Those handoff surfaces are optional LakeCat/QueryGraph extensions, not future mandatory Iceberg behavior. Parts of them may inform future standard profiles, especially lineage receipt binding and catalog event streams, but the QueryGraph semantic import itself belongs above the catalog.

This gives a simple reader rule. If a concept describes the table's portable metadata truth, call it Iceberg. If it describes how LakeCat makes that truth durable, atomic, redacted, and replayable, call it LakeCat implementation. If it describes policy receipts, secure agents, ODRL interpretation, or raw credential exceptions, call it TypeSec governance. If it describes semantic graph import, OSI/Croissant/CDIF alignment, or agentic application workflow, call it QueryGraph. Only after

an optional surface proves useful across catalogs and engines should it be described as a future Iceberg proposal.

The same rule can be applied as a release ledger.

Rust service/catalog spine: This is LakeCat implementation. Standard Iceberg does not prescribe a runtime, language, process layout, or dependency graph. The standard concern is that the catalog can resolve namespaces and tables, return metadata, and commit compatible updates. LakeCat's Rust spine exists so the standard request can be bound to identity, tenancy, store CAS, Sail planning, TypeSec receipt capture, audit, outbox, and replay validation in one local control path. It is not an Iceberg extension. A future Iceberg proposal should not say "use Rust"; it could say that a catalog may expose deterministic commit, scan, credential, and event proof produced by whatever implementation it uses.

Turso-backed local store: This is LakeCat implementation behind a portable store contract. Iceberg parlance talks about a catalog, a current metadata location, and atomic pointer updates. It does not require one durable database. LakeCat uses the Rust `turso` crate for the local spine because projects, warehouses, storage profiles, namespaces, tables, idempotency records, pointer logs, audits, and outbox rows need durable local transactions. Turso is therefore not an Iceberg extension and not a future Iceberg feature. The candidate future feature is a behavioral profile for catalog-state durability: atomic CAS, idempotent retry, redacted conflict evidence, pointer-history reads, and transactional event emission.

Iceberg REST-compatible table and namespace paths: These are standard Iceberg catalog parlance. A Spark, Flink, Trino, PyIceberg, or PySpark workflow should be able to create namespaces, load tables, and commit metadata through the standard catalog path without knowing that QueryGraph exists. LakeCat's standard path is the compatibility promise. The LakeCat and QueryGraph additions begin only where the client asks for management proof, replay verification, OpenLineage projection, governed scans, credential posture, or bootstrap bundles.

Commit CAS: This is standard catalog behavior with LakeCat hardening. Iceberg expects an optimistic commit to advance the metadata pointer only when the expected requirements still hold. LakeCat makes that atomic movement redacted, idempotent, audited, and replayable. The standard word is commit. The LakeCat mechanism is create-only metadata writes, store CAS, normalized idempotency, pointer-log evidence, audit rows, and outbox emission. The future candidate is an optional REST profile for idempotent commit replay and pointer-history proof.

Idempotency: This is LakeCat implementation today and a plausible future catalog profile. Iceberg clients retry commits, but the table format does not define every catalog's idempotency-key semantics. LakeCat treats idempotency as part of catalog safety: exact retry returns the stored response, drift under the same key conflicts, and replay does not duplicate graph or lineage side effects. It should not be written into table metadata. If standardized later, it should be an optional catalog behavior profile that ordinary clients can adopt without changing table files.

Pointer logs: This is LakeCat management evidence around standard pointer movement. Iceberg metadata already records table snapshots and metadata history, but a catalog pointer log records the catalog transaction that made one pointer current: principal, request hash, response hash,

idempotency-key hash, optional policy hash, and sequence. That is not standard Iceberg parlance today. It is an optional LakeCat management surface and a strong future candidate for interoperable catalog-history inspection.

Audit, outbox, and replay validation: These are LakeCat control-plane extensions. Audit explains authority. Outbox ties committed catalog facts to graph and OpenLineage delivery. Replay validation refuses to acknowledge or project malformed durable evidence. None of this belongs inside Iceberg metadata. The future Iceberg-adjacent idea is an optional catalog event stream with stable event identity, redaction rules, replay ordering, and lineage binding.

Governed scan planning: This is a LakeCat/TypeSec governance extension around standard Iceberg scan planning. Standard Iceberg gives engines the metadata needed to plan files. LakeCat adds policy-derived restrictions before planning: allowed columns, mandatory predicates, purpose, TTL, principal, and receipt proof. Sail should perform the Iceberg planning because it understands schemas, field ids, manifests, metrics, deletes, and format evolution. A future proposal might define an optional proof-carrying scan profile, but the base table remains a normal Iceberg table.

Credential paths: Credential vending exists in catalog practice and Iceberg REST discussions, but LakeCat deliberately narrows it. Trusted principals can receive audited raw credential exceptions when policy allows them. Agents and fine-grained restricted principals should be steered to Sail-planned reads instead of broad object-store access. The standard idea is catalog-mediated access. The LakeCat/TypeSec addition is credential-root proof, block reason proof, and receipt evidence. The future candidate is a governed credential-vending profile that standardizes how catalogs prove why credentials were issued or withheld.

QueryGraph and QGLake handoff: This is not standard Iceberg. It is an optional LakeCat/QueryGraph integration surface over standard catalog facts. LakeCat exports the evidence anchors: namespaces, tables, views, current pointers, commit proof, scan proof, credential posture, management inventory, OpenLineage hashes, and replay verification. QueryGraph owns Croissant, CDIF, OSI, ODRL, semantic graph import, agent workflows, and user-facing reasoning. Some components, especially lineage receipt binding and catalog event streams, may become future interoperability profiles. The QueryGraph semantic bundle itself should remain an application extension.

1.6 A Field Guide To The Catalog Concepts

The first confusion in any advanced catalog design is vocabulary. Some words come from Iceberg. Some describe how a particular catalog is implemented. Some describe governance and semantic systems that sit beside the catalog. If those categories blur, the architecture becomes either too timid or too invasive. A timid catalog becomes a passive pointer map. An invasive catalog starts inventing private table semantics and breaks the ecosystem it is trying to serve.

LakeCat uses a stricter rule. A concept belongs to standard Iceberg when it is part of the portable table or REST catalog contract. A concept belongs to LakeCat implementation when it is how this Rust catalog satisfies that contract reliably. A concept is a LakeCat or QueryGraph extension when it adds optional control-plane value beside the standard path. A concept belongs to TypeSec when

it describes authorization semantics, capability proof, secure-agent posture, or policy-derived restrictions. A concept becomes a possible future Iceberg proposal only after it has proven useful as an optional, interoperable profile that other catalogs and engines could adopt without changing the table format.

That gives each current LakeCat claim a precise status.

Rust service/catalog spine exists: This is LakeCat implementation. Iceberg does not prescribe Rust, async Rust, Axum, a crate layout, or an in-process planning path. It prescribes catalog behavior: resolve namespaces and tables, return metadata, enforce optimistic commit requirements, and speak compatible REST shapes. LakeCat chooses Rust because the catalog needs to keep request identity, tenancy, CAS, idempotency, Sail planning calls, TypeSec receipts, audit writes, outbox writes, and replay validation in a single strongly typed control path. That is a performance and correctness decision, not an Iceberg extension. The future Iceberg-adjacent lesson is behavioral: catalogs should be able to prove what they committed, planned, vended, and emitted, regardless of implementation language.

Turso-backed local store direction is in place: This is also LakeCat implementation. The Iceberg word is not "Turso"; the Iceberg words are catalog, table identifier, metadata location, and atomic commit. LakeCat uses the Rust `turso` crate as the durable local spine behind `CatalogStore` because the catalog needs local transactions for warehouses, storage profiles, namespaces, tables, views, idempotency records, pointer logs, audit rows, and outbox rows. Turso should not be proposed as an Iceberg feature. The portable part is the store contract: atomic compare-and-swap, exact idempotency replay, conflict rejection for drift, stable redaction, durable pointer history, audit evidence, and transactional event emission.

Iceberg REST-compatible table and namespace paths exist: This is standard Iceberg catalog parlance. It is the compatibility line. A Spark, PySpark, Trino, Flink, or PyIceberg client should be able to use the standard catalog surface to create namespaces, load tables, and commit metadata updates without knowing that QueryGraph exists. LakeCat can add management, bootstrap, replay, OpenLineage, graph, and credential-proof APIs beside that surface, but those APIs must not be hidden prerequisites for ordinary table access. If a client needs QueryGraph semantics to perform a normal Iceberg read, LakeCat has crossed the wrong boundary.

Commit CAS is heavily hardened: The standard Iceberg concept is the optimistic catalog commit. A writer has a base table state, writes new metadata, and asks the catalog to advance the current metadata pointer only if the expected requirements still hold. LakeCat keeps that behavior and hardens the implementation around it. The store does the atomic compare-and-swap. The metadata write path is create-only. The idempotency record ensures exact retry has one effect and drift has none. The pointer log records the accepted transition. Audit records authority. The outbox records the durable fact for graph and lineage. Replay validation checks that committed evidence still agrees before delivery. The Iceberg term is commit. The LakeCat terms are idempotency, pointer log, audit, outbox, replay validation, and redacted proof. A future Iceberg proposal could define an optional idempotent commit-replay profile, but it should not change the table metadata file.

Idempotency is a catalog reliability feature: Iceberg engines retry. Networks fail. Clients may not know whether a commit response was lost after the catalog accepted the pointer movement. LakeCat treats idempotency as a first-class safety contract. A repeated request under the same idempotency key and same body receives the stored response. A different body under the same key is rejected. A replayed success does not emit a second graph event, OpenLineage event, audit story, or pointer-log transition. This is not standard Iceberg table metadata. It is implementation behavior that could become an optional REST catalog convention.

Pointer logs are catalog history, not table history: Iceberg metadata already contains snapshots and metadata history. That history answers what the table says about itself. LakeCat's pointer log answers what the catalog did: which transaction advanced which pointer, under which principal, with which idempotency-key hash, request hash, response hash, policy hash, and sequence. Those two histories complement each other. The table history remains portable Iceberg metadata. The pointer log is LakeCat management evidence. A future optional catalog-history profile could standardize the shape without requiring every table metadata file to embed deployment-specific audit state.

Audit, outbox, and replay validation are hardened: These are LakeCat control-plane mechanisms. Audit records who or what acted. The outbox makes graph and OpenLineage side effects depend on committed catalog state rather than best-effort handler callbacks. Replay validation checks the durable payload before acknowledgement, including principal shape, table identity, hash form, counts, read restrictions, credential posture, management actor evidence, and receipt context. These mechanisms are intentionally outside Iceberg metadata. They are excellent candidates for future optional catalog event profiles because many catalogs need trustworthy event emission, but the standard table should remain readable by clients that know nothing about LakeCat's replay validators.

Governed scan paths carry TypeSec-style receipt evidence: The standard Iceberg work is scan planning over metadata: bind expressions, apply projection, prune manifests and files, attach delete files, and produce tasks for execution. LakeCat adds a governed prelude. It identifies the principal and purpose, asks TypeSec for a decision, derives allowed columns, mandatory predicates, TTL caps, and raw-credential posture, and asks Sail to plan the effective request. LakeCat then records both the user request and the allowed request as receipt evidence. The scan remains a scan over a normal Iceberg table. The proof that the scan was narrowed by policy is a LakeCat/TypeSec governance extension. A future Iceberg-adjacent proposal could standardize proof-carrying scan planning, but only as an additive profile.

Credential paths are governed access decisions: A catalog may help clients obtain storage access. LakeCat narrows the meaning of that power. Trusted principals can receive audited raw credentials when policy allows it. Agents and restricted principals should normally receive Sail-planned work instead of broad object-store power. The TypeSec receipt explains whether raw credentials were allowed, blocked, or replaced by a governed plan, and LakeCat validates that evidence before projecting it to graph or OpenLineage. The standard idea is catalog-mediated access. The LakeCat/TypeSec addition is proof of why access was issued, withheld, or converted into a governed Sail plan.

QueryGraph and QGLake handoff surfaces are broad: These are optional LakeCat/QueryGraph extension surfaces. QueryGraph needs a rich bootstrap story: warehouses, namespaces, tables, views, current pointers, commit history, view receipt chains, credential posture, scan proof, OpenLineage receipts, management inventory, Croissant/CDIF/OSI/ODRL context, and replay-verification results. LakeCat should not make those concepts part of the Iceberg table. LakeCat should export catalog facts and proof anchors. Grust should own graph taxonomy, storage, traversal, and query behavior. TypeSec should own security semantics. QueryGraph should compose those facts into the semantic application graph and agent workflow. Some handoff pieces, especially lineage receipt binding and catalog event identity, may later inform standards work. The QueryGraph semantic bundle itself is an application extension.

OpenLineage projection is a consumer of catalog truth: OpenLineage is not Iceberg, but it becomes much more useful when it is emitted from the catalog's durable outbox. A table commit, governed scan, credential decision, view change, storage-profile update, or management mutation should not become lineage merely because a handler tried to emit an event. It should become lineage because the catalog transaction committed and the replay validator accepted the durable evidence. That is LakeCat's extension. A future interoperability profile could describe how catalog events bind to OpenLineage receipts, but ordinary Iceberg metadata should stay untouched.

Management and view proof surfaces are operational extensions: Warehouses, projects, storage profiles, policy bindings, servers, view receipt chains, soft-delete state, and compact management inventories are not the core portable Iceberg table format. They are the operational reality around a catalog. LakeCat exposes them as management and proof surfaces, validates their replay payloads, and lets QueryGraph import them as governed catalog context. The future standards question is selective: view lifecycle proof, pointer-history inspection, and event-stream identity may deserve optional profiles. Project naming, local storage-profile rows, and the exact management schema are LakeCat implementation choices.

The most important architectural implication is that LakeCat should not turn standard compatibility into local reimplementation. Anything that understands Iceberg table structure belongs as far down in Sail as possible. Manifest metrics, delete-file association, schema and partition evolution, field-id binding, metadata-as-data, v3 row lineage, v4 metadata trees, branch and snapshot selection, and commit requirement validation are engine-shaped. LakeCat should initiate those actions from a governed catalog transaction, but Sail should perform the reusable table-format work.

That is why Sail is a strong engine choice. It is Rust-native, close to Arrow and DataFusion, already shaped around generated Iceberg REST models and table providers, and reusable by more than LakeCat. A governed PySpark workflow can still see a normal REST catalog while LakeCat validates and records the transaction. A governed agent can be denied raw credentials and receive a Sail-planned task set. A QueryGraph bootstrap can consume catalog evidence whose table facts were interpreted by the same engine path that will serve future Rust lakehouse execution. Sail lets LakeCat stay thin without staying blind.

The extension rule is therefore conservative:

1. Do not call implementation details Iceberg extensions.
2. Do not store LakeCat, QueryGraph, TypeSec, graph, or OpenLineage state inside Iceberg table metadata when it can live beside the table.
3. Keep optional LakeCat/QueryGraph APIs additive and discoverable.
4. Promote a feature toward an Iceberg proposal only when the behavior is useful across catalogs and engines and can be standardized as an optional profile.
5. Push reusable table-format work into Sail before adding a LakeCat-local parser, planner, or validator.

1.7 The Current Catalog State

The current lakehouse catalog market is converging on a simple fact: the catalog is no longer a sidecar. It is the place where table identity, tenancy, commits, credentials, governance, and interoperability are negotiated.

Hive Metastore gave early lakehouses a familiar namespace and table registry, but it was born for a different format era. Cloud warehouses built proprietary catalogs around their own engines. Nessie explored Git-like table references and branching. Unity Catalog turned governance and sharing into a central product surface. Lakekeeper has shown how a modern open Iceberg REST catalog can model warehouses, projects, credentials, soft deletion, and management APIs without polluting table metadata. Polaris has made the strongest standards-centered move: an open Iceberg REST catalog with vendor gravity, clear governance ambition, and a credible route for many engines to meet at the same catalog boundary.

That is why Polaris is a winner. It is not because it is the last word in catalog architecture. It is winning because it occupies the obvious shared surface at the right moment:

- It speaks Iceberg REST instead of asking every engine to learn a new table protocol.
- It treats the catalog as a security and governance surface, not merely a pointer map.
- It gives enterprises an open center of gravity around Iceberg while the warehouse, query-engine, and cloud-storage layers remain plural.
- It can be adopted incrementally: engines can keep reading tables, while operators gain a real control plane.

LakeCat should learn from that. The winning move is not to reject Polaris-style compatibility. The winning move is to keep that compatibility and then ask what a Rust-native, QueryGraph-facing catalog can do when it is allowed to plan near Sail, emit graph through Grust, and ask TypeSec for authorization proofs.

1.8 What Intermediation Loses

Catalogs win by sitting between engines and data. That same position can also flatten the system.

When the catalog is only an intermediary, it often sees the table name, the current metadata pointer, the caller, and perhaps a credential request. The engine sees the schema, manifests, statistics, deletes, partition evolution, scan filters, and physical plan. The governance system sees policy. The lineage system sees an event after the fact. The semantic layer sees a separate model. Each system receives a shard of the truth.

The loss is not merely elegance. It is operational:

- Policy can be checked before access but not carried into scan planning.
- Credentials can be vended without proving why a raw credential exception was allowed.
- Lineage can describe that something happened but not bind to the exact governed plan, snapshot, policy, and table metadata.
- Semantic layers can drift from the physical tables they describe.
- Agents can receive broad storage access when they should have received a governed plan and short-lived, narrow task set.
- Engines can optimize with file statistics while catalogs remain blind to the consequences of those choices.

The catalog should not become a replacement engine. But it should not be a blind intermediary either. LakeCat's thesis is that the catalog can stay thin and standard while still becoming engine-close at the moments where correctness, security, and semantics depend on planning evidence.

1.9 Why Sail Matters

Sail is the Rust lakehouse engine path. It already contains the pieces that should understand Iceberg deeply: catalog abstractions, generated Iceberg REST models, DataFusion table providers, manifest pruning, metadata-as-data paths, write and commit plumbing, and format-version checks.

That matters because scan planning and commit validation are not generic catalog chores. They require knowledge of Iceberg schemas, projections, partition specs, sort orders, snapshots, manifests, data files, delete files, statistics, and expression models. If LakeCat reimplements those details, it becomes a second Iceberg engine. The same bug class appears twice, and the catalog begins to drift away from the planner that actually executes work.

LakeCat takes the opposite path. Put reusable Iceberg and planning behavior in Sail. Let LakeCat call Sail for the parts that require engine-grade knowledge. Keep LakeCat responsible for the catalog boundary: identity, tenancy, durable state, policy gates, idempotency, audit, outbox, and standard REST compatibility.

The current LakeCat architecture follows this line. The service exposes an Iceberg REST-compatible surface under `/catalog/v1`. The Sail-facing engine path handles scan planning, table-status conversion, metadata preparation, manifest expansion, and standard response validation. LakeCat keeps only the additional context it must own: which principal asked, which policy narrowed the read, which metadata pointer was current, which commit won, and which event should be replayed to graph and lineage sinks.

1.10 Why Move Catalog Work Closer To The Engine

A passive catalog returns metadata locations and lets the client plan. That is compatible, but it is not enough for the next generation of governed systems. Agents, notebooks, services, and model pipelines need stronger guarantees:

- A policy should be enforced before a scan is planned.

- Column restrictions should narrow the projection before file tasks are produced.
- Row predicates derived from policy should become mandatory scan filters.
- A stateless `fetchScanTasks` call should not widen a prior governed plan.
- Credentials should be short-lived, scoped, and audited.
- Commit retries should be idempotent and should not replay a different request.
- Graph and lineage side effects should reflect committed catalog state, not best-effort handler side effects.

Those guarantees sit between catalog state and engine planning. If the catalog is too far from the engine, it can check a policy and then hand the client a metadata pointer, hoping the client preserves the restriction. If the engine is too far from the catalog, it can plan efficiently but may not know the governance receipt or durable identity context.

LakeCat fuses those two moments. LakeCat authorizes the request, derives the effective restriction, and asks Sail to plan or validate against the current metadata pointer. Sail performs the Iceberg work. LakeCat returns the standard shape and records the governance evidence.

The result is still an Iceberg catalog. The difference is that governed reads and writes are planned through the same Rust implementation that QueryGraph will use downstream.

The deeper reason is that Iceberg correctness is engine-shaped. A catalog can store a metadata pointer and compare it during commit, but the hard questions are about the table described by that pointer:

- Which schema id is current, and how do projected field ids map through evolution?
- Which partition spec applies to a manifest, and can a predicate be evaluated against its lower and upper bounds?
- Which delete files must travel with a data file task?
- Which statistics are absent, stale, truncated, or encoded with format-specific rules?
- Which snapshot or branch should a read bind to?
- Which update requirements conflict with the current metadata?
- Which metadata tables expose the right diagnostic view without forcing a client to parse every object manually?

Those are not good catalog-only questions. They are Iceberg engine questions. If LakeCat answers them locally, it must reimplement expression binding, schema projection, manifest reading, metrics decoding, delete semantics, format-version checks, and table-status conversion. The catalog would slowly become a second planner with a smaller test surface than the real engine. That is exactly the shape LakeCat should avoid.

Sail is the better place for that work for six practical reasons.

1. Sail is already Rust-native. LakeCat can call it without crossing a JVM service boundary, serializing every intermediate object through a foreign adapter, or hiding engine evidence behind opaque text blobs.

2. Sail sits in the Arrow and DataFusion ecosystem. Planning can produce structures that are natural for columnar execution, metadata-as-data queries, table providers, and future in-process QueryGraph workflows.
3. Sail already carries Iceberg-specific code paths: generated REST models, catalog provider seams, table-status conversion, manifest expansion, pruning helpers, write plumbing, and format-version checks.
4. Sail can be tested once and reused by more than LakeCat. If manifest metric decoding, delete-file indexing, or v4 metadata-tree support is improved in Sail, LakeCat, QueryGraph, and other Rust lakehouse tools all benefit.
5. Sail is close enough to execution to understand cost and shape. A catalog can know that a policy narrowed a scan, but the engine knows how that projection changes files, tasks, columns, delete handling, and downstream execution.
6. Sail lets LakeCat keep the compatibility promise. LakeCat can expose normal REST responses while asking Sail for the engine-grade validation and plan evidence required by governed reads and commits.

That gives LakeCat a concrete rule for future work. If a feature has to understand Iceberg metadata structure, expression binding, file statistics, delete semantics, snapshot selection, schema evolution, partition evolution, sort order, row lineage, metadata tables, or physical task shape, LakeCat should first try to push it into Sail. The catalog may initiate the work and persist the receipt, but the reusable implementation should live where the engine can use it too.

The following responsibilities are Sail-shaped:

- decoding manifest metrics and using them for pruning;
- interpreting lower and upper bounds across Iceberg physical encodings;
- expanding manifest lists into file scan tasks;
- attaching positional and equality delete files to the data tasks they affect;
- binding REST expressions to table schemas and field ids;
- preserving nested field projection through schema evolution;
- validating commit requirements against current table metadata;
- preparing new metadata JSON for commits, creates, deletes, and restores;
- exposing metadata-as-data tables for snapshots, manifests, files, history, and partitions;
- carrying v3 row-lineage and v4 metadata-tree behavior once those models are typed in Sail;
- converting Iceberg REST and table metadata into engine table status and provider objects.

The matching LakeCat responsibilities are catalog-shaped:

- decide which principal, warehouse, namespace, table, and policy context apply;
- ask TypeSec for an authorization decision and receipt;
- derive the effective read restriction from policy and request context;
- call Sail with the narrowed projection, mandatory filters, purpose, and current metadata pointer;
- persist the metadata pointer only after the catalog transaction wins;
- store idempotency, audit, pointer-log, and outbox evidence;
- reject replay evidence that no longer matches the durable receipt;

- publish optional QueryGraph, Grust, and OpenLineage handoff surfaces.

That separation is important for both speed and correctness. A Rust catalog can call a Rust engine path directly, without a JVM service hop or a pile of language-adapter objects, but the benefit is deeper than latency. It means the same implementation that plans a governed task set for LakeCat can also be used by notebooks, QueryGraph ingestion, maintenance jobs, and future Rust execution surfaces. Every manifest-pruning fix, delete-file fix, or v4 metadata fix lands once in Sail and becomes available to the whole stack.

This division also improves security. A policy decision is not useful if it is only an annotation on a request. LakeCat should turn a TypeSec decision into an effective projection, mandatory predicate, purpose, and receipt. Sail should plan from that effective request, not from the client's wider request. LakeCat then records both sides: what was asked for, what was allowed, what Sail planned, which metadata pointer was current, and which receipt authorized the result. That is the evidence QueryGraph needs, and it is stronger than handing an agent an object-store credential and hoping the client behaves.

The same division improves performance. Pushing pruning, manifest expansion, delete handling, and metadata-as-data into Sail means LakeCat does not need to load and reinterpret Iceberg structures just to prove a governed task set. When the service and engine are both Rust, the catalog can make a local call, reuse typed objects, and persist compact proof evidence rather than copying large metadata bodies through multiple services. The fast path remains the standard Iceberg path for ordinary engines. The governed path becomes richer without becoming a compatibility tax.

Consider a governed read. A user or agent asks for a table, a projection, and perhaps a filter. LakeCat can identify the principal, warehouse, namespace, table, request purpose, and policy context. TypeSec can decide that the request is allowed only for certain columns, with a mandatory row predicate and a credential TTL cap. At that point LakeCat should not parse every manifest and invent file tasks itself. It should call Sail with the effective request. Sail can bind fields by Iceberg field id, preserve nested projection, evaluate partition statistics, attach delete files, and return task evidence that corresponds to the real table metadata. LakeCat records the receipt and hashes of that plan. QueryGraph can later verify that the agent saw a governed plan, not a broad pointer and a polite suggestion.

Consider a commit. LakeCat owns request identity, idempotency, compare-and-swap, audit, pointer logs, and outbox delivery. Sail should own the table-format part: checking update requirements against current metadata, preparing or validating new metadata JSON, understanding format-version behavior, and returning the standard response shape. If LakeCat writes the pointer only after Sail validates the table-format work, the catalog transaction remains authoritative without duplicating the engine. If the writer retries, LakeCat's idempotency record decides whether it is the same request. If the pointer moved, LakeCat returns a redacted conflict. If the commit wins, the outbox can project graph and lineage from a durable catalog fact.

Consider a QueryGraph bootstrap. LakeCat should provide catalog facts: warehouses, namespaces, tables, views, current pointers, commit history, credential posture, scan receipts, view receipt chains, and OpenLineage hashes. Sail should provide the engine facts behind those catalog

facts: what the metadata describes, what a plan contains, what delete files apply, and how newer Iceberg versions should be interpreted. Grust should own graph taxonomy and traversal. TypeSec should own policy meaning and secure-agent receipts. QueryGraph should own Croissant, CDIF, OSI, ODRL application semantics and the final graph import. That is how LakeCat can become foundational without becoming a warehouse, a policy engine, a graph database, and a semantic app at the same time.

In short: LakeCat should own trust, identity, pointers, transactions, and evidence. Sail should own Iceberg semantics, planning, pruning, metadata interpretation, and engine-facing execution shape. QueryGraph should consume the resulting evidence as a semantic graph. That is the architecture that keeps LakeCat thin without making it weak.

Sail is a particularly good engine choice because it is close to every object LakeCat should not duplicate. A governed read is not just a list of files. It is a bound expression over an evolved schema, a projection over field ids, a snapshot selection, a partition pruning problem, a manifest-metric decoding problem, and often a delete-file association problem. A governed commit is not just a pointer write. It is an update-requirement check, metadata JSON preparation problem, object-write plan, and response-shape validation problem. Those are precisely engine-shaped responsibilities.

If LakeCat implemented those details locally, each standard improvement would have to land twice. A new Iceberg metadata version would require catalog-side parsing and engine-side parsing. A metrics-decoding bug would need one fix in the planner and one fix in the catalog proof path. A delete-file attachment bug could be fixed for query execution while the catalog still produced stale governed task proof. That is not a thin catalog; it is an accidental second engine. Sail prevents that drift by becoming the reusable Rust place where the Iceberg semantics live.

The performance argument points the same way. LakeCat should be able to call a Rust Sail API with typed request state, receive typed plan or validation evidence, and persist compact hashes and counts. It should not shell out to a separate planner, round-trip through a JVM bridge, or translate every metadata structure through an untyped JSON corridor just to prove that a policy narrowed a scan. For ordinary engines, LakeCat can still return ordinary REST responses. For governed QueryGraph and agent paths, the same request can carry stronger proof because Sail has already done the real planning work.

Sail is also the right place because the user workflow begins long before QueryGraph sees a graph. A PySpark user may create a table and commit metadata through an Iceberg REST catalog. A notebook may ask for a scan. A maintenance job may inspect manifests. An agent may ask for a governed subset of the same table. QueryGraph may later import the table as a semantic asset. All of those flows depend on the same Iceberg facts: schemas, snapshots, manifests, partition specs, delete files, statistics, and metadata evolution. If each surface implements those facts independently, the system becomes inconsistent by construction. If Sail owns them, LakeCat can expose different catalog surfaces while depending on one reusable engine truth.

The PySpark path illustrates the point. Spark should see a normal Iceberg REST catalog. It should not care about QueryGraph bootstrap or TypeSec receipts when it performs ordinary table work.

But LakeCat still benefits from Sail because the catalog can validate standard request and response shapes against the same Iceberg model that the Rust engine uses. When a commit succeeds, LakeCat owns the transaction and evidence. Sail owns the table-format understanding. Spark gets compatibility, and LakeCat gets a trustworthy proof trail.

The governed-agent path is different but uses the same engine core. The agent should not receive the broad storage credential merely because a standard catalog can vend one. LakeCat asks TypeSec for a decision, derives an effective projection and row restriction, and asks Sail to plan the narrowed request. Sail can prune manifests, attach delete files, and shape scan tasks from the actual table metadata. LakeCat records what was requested, what was allowed, what Sail planned, which credential posture was chosen, and which receipt authorized it. QueryGraph can then verify the evidence without trusting the agent or re-planning the lake by itself.

The management path also benefits. Operators need to inspect pointer history, storage-profile posture, view receipt chains, credential decisions, and replay delivery status. Those are catalog facts, so LakeCat owns the API and store contract. But when an operator asks why a governed scan saw only a subset of files, the answer depends on engine facts: partition pruning, manifest metrics, delete handling, and snapshot selection. Sail gives LakeCat a way to explain that outcome without duplicating the implementation that produced it.

This is why pushing work into Sail is not merely an optimization. It is the mechanism that lets LakeCat stay both standard and ambitious. The catalog can remain narrow at the Iceberg boundary while becoming rich in evidence. The engine can remain reusable while carrying the hard table semantics. QueryGraph can build a semantic graph on top of proof rather than inference. TypeSec can make security decisions that affect real plans rather than annotations. Each part gets stronger because the table-format work is concentrated where it can be tested, optimized, and reused.

Sail is a strong engine choice for LakeCat because it matches the shape of the problem. Iceberg is metadata-heavy, columnar, versioned, and planner-driven. Rust is a good fit for a catalog that must be fast, explicit about ownership, careful with redaction, and comfortable passing typed objects between the service, store, and engine layers. Sail adds the engine side of that same discipline: generated Iceberg REST models, Arrow/DataFusion-native execution objects, catalog/provider seams, manifest and metadata paths, and a natural place to grow v3 and v4 table-format support. LakeCat can therefore keep the catalog transaction small while still asking a real engine to answer engine-shaped questions.

That choice changes user workflows without breaking them. A PySpark user still sees a normal Iceberg REST catalog. A Rust service can call the same catalog and receive evidence-rich governed planning. An agent can be denied raw credentials and handed a Sail-planned task set instead. An operator can inspect pointer logs and replay proof while knowing that file pruning and delete-file attachment came from the reusable engine path. QueryGraph can import a semantic graph whose catalog facts were produced by durable LakeCat state and whose table facts were interpreted by Sail. That is the practical meaning of pushing work into the engine: the standard path stays

portable, and the advanced path gets stronger because it is built on the same Iceberg semantics rather than a catalog-side approximation.

The design rule is therefore operational, not philosophical:

1. If the work needs table-format semantics, push it into Sail.
2. If the work needs catalog atomicity or durable request evidence, keep it in LakeCat.
3. If the work needs graph taxonomy, traversal, projection storage, or Cypher, push it into Grust.
4. If the work needs authorization semantics, capability composition, ODRL meaning, TypeDID envelopes, or secure-agent proof, push it into TypeSec.
5. If the work needs semantic application import, OSI/Croissant/CDIF alignment, or end-to-end graph acceptance, make it QueryGraph's responsibility and let LakeCat provide the catalog facts.

That rule makes first-release scope clearer. LakeCat needs enough Sail integration to prove that reads and commits are planned and validated through the engine path. It needs enough TypeSec integration to prove authorization receipts and governed restrictions. It needs enough Grust/OpenLineage output to prove replayable side effects. It needs enough QueryGraph handoff to prove that the next system can bootstrap from LakeCat without inventing a private import shortcut. It does not need to own every future semantic model, graph query, policy language feature, or Iceberg parser. The strongest catalog is the one that knows exactly when to stop and call the right sibling.

1.11 Catalog Concepts In User Workflows

The concept boundary becomes clearest when it is traced through ordinary user workflows. A catalog concept should not be classified by how advanced it sounds. It should be classified by the role it plays in the request. The same table can be touched by a PySpark job, a Rust service, a governed agent, an operator, a lineage consumer, and QueryGraph. Each client may see a different surface, but the portable table truth should remain Iceberg metadata and the engine-shaped table work should remain Sail.

The PySpark workflow is the compatibility baseline. A Spark user configures an Iceberg REST catalog, creates a namespace, creates a table, writes data, and later loads the table through the catalog. In standard Iceberg terms, that workflow uses namespaces, table identifiers, table metadata, snapshots, manifests, data files, delete files, current metadata locations, and optimistic commits. LakeCat should look like an ordinary Iceberg REST catalog for that flow. The user should not need to know about QueryGraph bootstrap, TypeSec receipts, Grust projection, OpenLineage receipt hashes, or Turso rows.

LakeCat still does real work during that ordinary PySpark flow. The Rust spine normalizes the request, resolves tenancy, persists namespace and table state, performs compare-and-swap on the metadata pointer, records idempotency when the client supplies a key, writes pointer-log and audit evidence, and enqueues committed events. Those are LakeCat implementation details around standard Iceberg behavior. They are not Iceberg extensions, because the PySpark client does not change its table model or call a private endpoint. They are the discipline that lets a standard catalog become inspectable and replayable.

Sail enters the PySpark story where the catalog would otherwise need table-format knowledge. If a commit needs metadata preparation or validation, or if LakeCat needs to validate standard response shape against Iceberg models, the reusable implementation belongs in Sail. The catalog should not grow its own partial manifest reader or metadata-version validator simply because the client happened to be Spark. Spark gets the standard surface. LakeCat gets the durable transaction. Sail keeps the table-format semantics reusable.

A notebook or data-service workflow is slightly richer. A Python or Rust service may ask the catalog to plan a read, fetch tasks, inspect metadata, or obtain short-lived access. In standard Iceberg terms, the engine still needs schemas, projections, partition specs, snapshots, manifests, statistics, delete files, and scan tasks. LakeCat can add request identity, purpose, allowed columns, required row predicates, credential TTL caps, and receipt evidence. The first group is Iceberg and engine work. The second group is LakeCat/TypeSec control-plane work.

That distinction matters because a governed notebook should not rely on a client promise to behave. If policy narrows the request to five columns and a mandatory row predicate, LakeCat records the requested projection and the effective projection, asks TypeSec for the receipt, and calls Sail with the effective request. Sail binds fields by Iceberg field id, handles schema evolution, evaluates partition and manifest statistics conservatively, attaches delete files, and returns a plan shape the catalog can expose. LakeCat then stores proof of the narrowed plan. The standard table is unchanged. The governed evidence is additive.

The governed agent workflow is the reason LakeCat cannot be only a passive pointer map. An agent may ask for data in order to answer a question, train a model, enrich a graph, or take an action. Giving that agent raw object-store credentials is often the wrong default. The safer path is to treat raw credential vending as an audited exception and treat Sail-planned reads as the normal governed path. LakeCat identifies the agent, asks TypeSec for a decision, derives a read restriction, asks Sail to plan against the current metadata pointer, and returns only the narrowed work or credential posture the policy allows.

In standard Iceberg parlance, the agent workflow still touches a normal table. The scan still comes from Iceberg metadata. The delete files still mean what Iceberg says they mean. The snapshots and manifests are not QueryGraph objects. The LakeCat/TypeSec additions are the receipt, purpose, TTL, allowed-column set, row predicate, raw-credential exception proof, block reason, and replay validation. Those additions are extensions around catalog access, not custom table metadata. They are good candidates for future optional governed-access profiles precisely because they can be described without changing the table.

An operator workflow exercises a different surface. Operators need to know which metadata pointer is current, which commits won, which retries were exact, which idempotency keys drifted, which credentials were issued or withheld, and which outbox events have reached graph and lineage sinks. Iceberg table metadata answers part of this story: it records table snapshots and metadata history. LakeCat pointer logs answer the catalog part: which transaction advanced the pointer, under which principal, request hash, response hash, idempotency-key hash, policy hash, and sequence. Audit and outbox rows answer authority and delivery questions.

Those operator surfaces should be viewed as LakeCat management extensions. They are not prerequisites for a standard client. They are not private fields inside metadata JSON. They are the catalog's operational memory. Some of them could become future Iceberg-adjacent proposals. Pointer-history inspection, idempotent replay profiles, and catalog event streams are broadly useful. The proposal shape should be optional REST or event profiles, not a requirement that every Iceberg table embed one deployment's audit vocabulary.

The lineage workflow shows why the outbox matters. If a table commit, scan, credential decision, or view update emits OpenLineage directly from the HTTP handler, lineage becomes best effort. The handler may fail after committing. The sink may be down. A retry may emit duplicates. LakeCat instead persists the catalog fact and outbox row in the transaction, validates the durable evidence on replay, then projects to OpenLineage. OpenLineage is not Iceberg, and it is not QueryGraph, but it becomes more trustworthy when it is bound to committed catalog state.

The graph workflow follows the same rule. LakeCat should not become a graph database. It should emit catalog-facing graph facts at the boundary: warehouses, namespaces, tables, views, commits, scans, credentials, management changes, and receipt anchors. Grust should own graph taxonomy, storage, projection logic, traversal, Cypher support, and graph query behavior. QueryGraph should compose the semantic application graph. In that workflow, Iceberg supplies the table truth, Sail supplies the engine interpretation, LakeCat supplies durable catalog evidence, Grust supplies graph mechanics, and QueryGraph supplies meaning.

The QueryGraph bootstrap workflow is therefore an integration flow, not a replacement catalog protocol. QueryGraph can ask LakeCat for a bootstrap bundle or QGLake handoff that includes warehouses, namespaces, tables, views, current pointers, commit proof, scan proof, credential posture, view receipt chains, management inventory, OpenLineage receipt hashes, and replay-verification results. QueryGraph can then align those facts with Croissant, CDIF, OSI, ODRL, and application semantics. That handoff is broad by design, but it remains an optional LakeCat/QueryGraph extension. Standard clients do not need it, and standard Iceberg metadata does not carry it.

This workflow view also answers whether LakeCat's ideas should be called extensions or future Iceberg features. The answer is intentionally split:

1. Rust service spine and Turso local store are implementation choices. They make LakeCat fast, durable, and inspectable, but they are not Iceberg extensions and should not become Iceberg features.
2. Iceberg REST namespace and table paths, current metadata pointers, manifests, delete files, snapshots, and optimistic commits are standard Iceberg. LakeCat should implement them faithfully.
3. Idempotency records, pointer logs, audit rows, outbox rows, replay validation, and management proof are LakeCat control-plane mechanisms. They are optional extensions around the catalog.
4. TypeSec receipts, secure-agent decisions, ODRL-derived restrictions, governed credential posture, and raw-credential exception proof are governance extensions.

5. QueryGraph bootstrap, Croissant/CDIF/OSI semantic import, Grust graph projection, and QGLake handoff are application and integration extensions.
6. Future Iceberg-adjacent proposals should be limited to behavior that is useful across catalogs and engines: idempotent commit replay, pointer history, catalog event streams, governed credential vending, proof-carrying scan planning, view lifecycle proof, and lineage receipt binding.

The common thread is that every extension stays additive. A PySpark job should keep working through the normal catalog path. A governed agent can opt into the proof-carrying path. An operator can inspect management evidence. QueryGraph can bootstrap a graph. The Iceberg table remains portable, and the strongest future proposals emerge from evidence that has already worked in real workflows.

That is the practical argument for pushing work into the engine. If LakeCat plans scans itself, it risks creating private semantics for every workflow. The PySpark path and the governed-agent path might disagree about field ids, partition pruning, delete files, v4 metadata interpretation, or expression binding. If Sail owns those semantics, the ordinary and governed paths share one engine truth. LakeCat can add identity, policy, receipts, transactions, audit, outbox, graph, and lineage without becoming a second table engine.

Sail is a particularly strong engine choice for that role because it is already the Rust side of the lakehouse. It is close to generated Iceberg REST models, Arrow/DataFusion execution objects, metadata-as-data, manifest handling, provider abstractions, and future v3/v4 format support. It can be improved once and reused by LakeCat, QueryGraph, notebooks, maintenance tools, and any other Rust lakehouse surface. That is how LakeCat can be ambitious without becoming invasive: the catalog owns trust and state, Sail owns table semantics, TypeSec owns policy meaning, Grust owns graph behavior, and QueryGraph owns the end-to-end semantic application.

1.12 LakeCat's Thin Boundary

LakeCat should be thin, but thin does not mean trivial. It owns the durable catalog state that must be correct even when external sinks are unavailable.

The core LakeCat responsibilities are:

- Serve the Iceberg REST Catalog API for standard clients.
- Model projects, warehouses, namespaces, tables, views, and storage profiles.
- Persist metadata pointers and compare-and-swap commit history.
- Validate idempotency keys and replay only matching commit bodies.
- Resolve request identity from headers, bearer tokens, agents, and TypeDID envelopes.
- Ask TypeSec for authorization decisions and persist receipts.
- Route scan planning and commit preparation through Sail.
- Record audit and outbox events inside the catalog transaction.
- Drain committed events to Grust and OpenLineage sinks.
- Publish a QueryGraph bootstrap bundle.

The deliberately excluded responsibilities are just as important:

- LakeCat does not invent a table format.
- LakeCat does not fork Iceberg manifest pruning.
- LakeCat does not own graph traversal or graph query behavior.
- LakeCat does not own security semantics or agent trust semantics.
- LakeCat does not author QueryGraph's final business semantic model.

That boundary gives each sibling project a clear job. Sail owns reusable Iceberg and planning behavior. Grust owns graph schema, storage, and query mechanics. TypeSec owns policy, capabilities, TypeDID envelopes, secure agents, and authorization semantics. QueryGraph owns the semantic application built on top.

1.13 The Read Path

The LakeCat read path begins like a standard catalog request and ends with a governed Sail plan.

1. A client asks to load or plan a table through the Iceberg REST surface.
2. LakeCat resolves the warehouse, namespace, table, and current metadata pointer.
3. LakeCat resolves the request identity.
4. LakeCat asks TypeSec whether the principal can perform the requested action.
5. TypeSec returns a decision and any enforced restrictions.
6. LakeCat turns those restrictions into a `ReadRestriction`: allowed columns, required row predicate, purpose, policy hash, and audit context.
7. LakeCat asks Sail to plan against the current metadata pointer with the effective projection and filters.
8. Sail validates Iceberg expressions against generated REST models and table schema, expands manifests, applies conservative file-bound pruning when metrics are present, and carries delete-file references into file scan tasks.
9. LakeCat returns Iceberg-compatible plan and task responses.
10. LakeCat records audit and outbox events that can later be projected into graph and lineage.

The important detail is that the policy restriction becomes part of planning, not a note beside it. An empty client projection under a column restriction means the allowed columns. A client projection can narrow further, but cannot widen. LakeCat records both the client's requested projection and the effective projection that survived the server-derived column restriction in the durable scan-planned replay evidence. The same rule applies to stats-field requests: LakeCat records both the client's requested stats fields and the effective stats fields that survived the restriction, while the compatibility `stats-fields` extension remains the narrowed effective set. The default REST path is tested at the Sail boundary: Sail receives only the effective projection and mandatory policy filters, while LakeCat keeps the broader request and narrowed result as replay evidence. During `fetchScanTasks`, LakeCat recomputes the current restriction and sends Sail the required projection and mandatory filters again; the response extension and audit outbox record the same proof. A stale or legacy token cannot silently expand back to all columns. Outbox admission also checks that governed planned/fetched scan replay carries the same `read-restriction` in the top-level payload and in `authorization-receipt.context.read-restriction`, so replay cannot claim policy narrowing that the durable receipt did not capture. The same admission boundary requires governed scan replay to keep a nonblank `purpose` and a positive `max-credential-ttl-`

`seconds` value before graph or OpenLineage projection, so a QGLake handoff cannot learn task evidence whose purpose or credential TTL cap was lost before replay.

1.14 The Commit Path

The write path follows the same principle: LakeCat owns the catalog transaction, Sail owns reusable Iceberg validation and metadata preparation.

1. A client sends an Iceberg commit request, optionally with an idempotency key.
2. LakeCat validates the request shape and the idempotency key.
3. LakeCat resolves identity and asks TypeSec for the commit capability.
4. If the idempotency key already has an exact stored response, LakeCat returns that response before Sail validation or metadata-object writes.
5. LakeCat loads the current metadata pointer from the store.
6. LakeCat delegates Iceberg update validation and metadata assembly to Sail.
7. LakeCat rejects metadata-object writes that target the table's current metadata pointer, so the current metadata file cannot be overwritten before the store commit has won.
8. LakeCat rejects metadata-write plans that do not carry a concrete new metadata location.
9. LakeCat rejects metadata-object locations outside the table's matched storage profile prefix, and also rejects the storage-profile root itself because metadata commits must create a child object.
10. LakeCat rejects literal or percent-encoded dot path segments in metadata object locations, so commit plans cannot rely on traversal-like spelling to address anything other than a plain child object.
11. LakeCat writes the new metadata object through the warehouse storage profile with create-only object-store semantics.
12. LakeCat advances the table pointer with compare-and-swap.
13. LakeCat persists idempotency, audit, pointer-log, and outbox records.
14. If the store rejects the commit after a local metadata write, LakeCat cleans up the uncommitted metadata object when it can do so safely.
15. Outbox draining projects the committed event to graph and lineage sinks.

The cleanup path is deliberately secondary to the commit result. If metadata cleanup fails after the store rejects a commit, LakeCat preserves the original store or compare-and-swap error class and appends cleanup context. A stale pointer conflict still looks like a conflict to an Iceberg client, but the message carries SHA-256 hashes of the expected and actual metadata locations so operators can diagnose the race without exposing raw object paths. The service-level regression for this path checks the API response and the filesystem side effect together: the rejected metadata object is gone, while the conflict body contains only hashed pointer evidence. True cleanup failures use the same redaction discipline: the cleanup context identifies the uncommitted metadata object by `metadata-location-hash=sha256:...`, not by the raw object path. When that cleanup failure is appended to the preserved commit conflict, LakeCat keeps only `error-detail-hash=sha256:...` evidence for the cleanup detail, so raw backend text cannot leak through the combined conflict message. If cleanup discovers the uncommitted object is already absent, LakeCat treats that as successful cleanup rather than turning a resolved orphan into an internal error. Cleanup also

refuses to delete the previous committed metadata pointer if a future plan accidentally reports it as the staged write; the committed metadata object remains the table's current state, not an orphan. The same audit-safe shape applies before the write: current-pointer overwrite, existing-object overwrite, unsupported object-store configuration, and storage-profile-prefix failures report metadata-location hashes, and prefix mismatches also report a storage-profile-prefix hash rather than raw object paths. LakeCat also keeps the storage-profile id out of this error text, so tenant or profile naming conventions do not leak when a planned metadata object falls outside the selected root. A root-targeted metadata write uses the same redacted error shape: the operator sees that the plan did not name a child metadata object without receiving the raw table or storage root. Dot-segment failures use the same style: literal `..` and percent-encoded `%2e%2e` paths fail before object-store writes and expose only the metadata-location hash. Decorated metadata object locations with URI query strings, fragments, or URI userinfo are rejected at the same pre-write boundary, so a commit plan cannot smuggle version selectors, backend hints, fragment markers, or embedded credentials into what should be a plain metadata object address.

Idempotency is part of correctness. Reusing the same key for the same commit can return the stored response even after the table has advanced beyond the original commit requirements. Reusing the same key for a different body must conflict. LakeCat persists a normalized request hash and stores only audit-safe evidence, not raw secrets or raw idempotency keys. REST idempotency keys are intentionally narrow: `x-lakecat-idempotency-key` must be 1 to 128 ASCII characters and may use only letters, digits, `-`, `_`, `.`, or `:`. Invalid keys, including non-ASCII or invalid header bytes, fail before LakeCat performs authorization, Sail validation, table loading, or metadata-object writes. When a reused key is attached to a different commit body, the conflict response also stays redacted: it does not echo the raw key or the mismatched metadata object location. The Turso spine pins the same redaction for both commit-time reused-key conflicts and explicit idempotency replay probes: the raw key, mismatched request hash, and mismatched metadata object location are not operator-facing error text. The service regression for this path proves the replay happens before metadata-object writes: an exact retry returns the stored response without touching the already committed metadata object. The same regression pins the outbox side effect: exact replay and mismatched reused-key conflicts leave only the original `table.commit` outbox event, so QueryGraph and OpenLineage replay do not see duplicate commit work from retry traffic. Another regression sends a commit whose requested metadata location is the table's current pointer and verifies that LakeCat returns a bad request without touching the existing metadata file. Another sends a commit to a different metadata location that already exists and verifies that LakeCat returns a conflict without overwriting that non-current object. The same guard fails closed if a future Sail plan asks LakeCat to write metadata but does not provide a new object location, or if it tries to use the storage profile root as the new metadata object. A companion regression rejects both literal and percent-encoded dot path segments in a planned metadata location. When a backend object store fails setup, create-only write, or cleanup, LakeCat keeps the metadata location hash and adds hash evidence instead of returning raw backend text. Invalid metadata URI parsing and unsupported backend setup failures use `backend-error-hash=sha256:...`, making the setup-admission boundary explicit. Create-only write and cleanup failures keep `error-detail-hash=sha256:...` because those happen after setup. In every case,

the response names the hashed metadata location and hashed failure detail, not the submitted path, object name, scheme, or parser/backend diagnostic. That route-level promise is pinned by a commit regression for decorated metadata locations with raw query-token material. It matters for local files, cloud bucket keys, and credential-provider diagnostics: operators can correlate a failure without copying sensitive storage topology into API responses or logs.

The embedded in-memory store follows the same commit evidence contract as the Turso path. A successful commit emits one `table.commit` audit/outbox event with the compact commit record, authorization receipt, response hash, and redacted idempotency-key hash. An idempotent replay returns the stored response without adding a second outbox event, so tests and local embedded deployments exercise the same outbox invariant as the durable spine.

Commit records also carry a response hash over the stored table response. That pair matters: the request hash proves which commit body won or replayed, while the response hash proves which metadata pointer and table body LakeCat returned to clients and later projected through graph and lineage replay.

The same commit record includes compact summary evidence: Iceberg format version, current snapshot id, and the policy hash from the authorization receipt when one exists. QueryGraph can inspect those fields from the pointer-log/outbox stream without parsing full table metadata for every catalog audit question. Before a `table.commit` outbox event is projected or acknowledged, LakeCat now checks that it carries a commit object, an unsigned sequence number, a decodable root table identity, matching nested commit-table identity when present, both the commit principal and authorization receipt principal with matching values, and full `sha256:`-prefixed 64-hex request, response, idempotency-key, and present policy hashes. A prefix-shaped placeholder, contradictory commit identity, missing receipt principal, or drifted principal cannot become delivered commit replay evidence.

Operators and QueryGraph can read that pointer-log evidence through a governed management endpoint:

```
curl -s \
  -H 'x-lakecat-principal: operator@example.com' \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/tables/
events/commits
```

The response contains compact commit records: sequence number, previous and new metadata locations, request hash, response hash, idempotency-key hash, Iceberg format version, current snapshot id, policy hash, principal, and a commit hash. The read itself enters the durable outbox as `table.commits-listed` and drains as lineage evidence plus catalog-facing `Commit` graph anchors keyed by table and sequence number. That gives audit tools and QueryGraph a Grust-visible pointer-log inspection trail without requiring direct access to the Turso catalog database or making LakeCat a graph query engine. QGLake acceptance now exercises this path directly: the fixture issues an idempotent no-op commit probe, reads the compact commit-history endpoint, verifies that the record preserves the table's Iceberg format-version and current snapshot summary, requires the compact request, response, idempotency-key, commit, and optional

policy hashes to be full `sha256:`-prefixed 64-hex digests, and then requires the lineage drain to replay `table.commits-listed` receipt hashes plus compact commit count, sequence-number, and commit-hash summary fields before the QueryGraph handoff is accepted.

1.15 The Durable Spine

LakeCat's durable local spine uses the Rust `turso` crate behind the `turso-local` feature. The store contract remains portable, but the local foundation is not SQLx. The important tables are not an application afterthought; they define the catalog's control-plane memory:

- projects and warehouses;
- storage profiles;
- namespaces and tables;
- metadata pointer log;
- idempotency records;
- soft deletes;
- policy bindings;
- audit events;
- outbox events.

Object storage remains the source of Iceberg metadata files. Turso stores the atomic pointer, management state, idempotency evidence, and event record. This mirrors the Iceberg catalog contract: metadata files describe the table; catalog state decides which metadata file is current.

Outbox draining is intentionally strict. LakeCat projects a batch to graph and lineage sinks first, then acknowledges the whole projected batch in the store. Embedded and Turso stores select the same pending prefix by sorting on `created_at,event_id` before applying the drain limit, so a small batch means the same replay set in either durable backend. The drain response and delivery acknowledgement follow that ordered prefix, leaving later pending events for a future drain. If projection fails, nothing is acknowledged. If the store reports that fewer events were acknowledged than LakeCat projected, the drain fails with an acknowledgement mismatch instead of returning a quiet partial success. That keeps retry and operator evidence honest when a concurrent drain or backend anomaly interferes with delivery accounting. The regression suite covers the uncomfortable middle case too: if the first event in a multi-event batch already projected to graph and lineage but a later event fails during lineage projection, LakeCat still acknowledges none of the events. Recovery starts from the committed outbox batch instead of from a half-delivered response. The drain also refuses unknown event types before any projection happens. A future or custom event stays pending until LakeCat knows how to project it, instead of disappearing behind an empty graph/lineage receipt. The drain also validates governed-read evidence before projection. If a pending event contains a `read-restriction.policy-hashes` array, it must be non-empty and each entry must already be a full `sha256:`-prefixed 64-hex digest. A readable placeholder such as `sha256:policy-name`, or an empty policy anchor array, fails the drain before graph or lineage sinks run and before the store can mark the event delivered, keeping malformed source evidence available for retry or operator repair instead of promoting it into a QGLake handoff. LakeCat now applies that same admission rule to `authorization-receipt.context.read-restriction.policy-hashes`, so the receipt kept for later proof cannot

preserve an empty or placeholder policy anchor while the top-level scan event looks valid. LakeCat also rejects both planned and fetched scan replay when the top-level `read-restriction` differs from `authorization-receipt.context.read-restriction`, so graph and OpenLineage evidence cannot drift from the TypeSec receipt that authorized the narrowed read. Planned and fetched scan replay must also carry nonblank purpose evidence and a positive policy-derived credential TTL cap before the outbox event can be acknowledged or projected. Table commit events receive the same treatment for compact commit receipt evidence: `request_hash`, `response_hash`, `idempotency_key_sha256`, and any present `policy_hash` must be full digests before the event can be projected or acknowledged.

1.16 Grust For Graph Concepts

Catalog events naturally form a graph. A server contains projects. A project contains warehouses. A warehouse contains namespaces and storage profiles that define credential roots. A namespace contains tables. A table has columns, snapshots, manifests, files, policies, commits, scan plans, principals, and lineage runs. QueryGraph needs that graph, but LakeCat should not become a graph database.

Grust owns the graph layer. It is the place for reusable graph taxonomy, projection builders, graph stores, traversal indexes, Cypher support, and typed or untyped graph operations. LakeCat's responsibility is narrower: translate committed catalog events into a bounded envelope and pass it through a catalog-facing sink.

In practice this means LakeCat can emit graph events for stable catalog facts:

```
Server CONTAINS Project
Project CONTAINS Warehouse
Warehouse CONTAINS Namespace
Namespace CONTAINS Table
Table HAS_COLUMN Column
Table GOVERNED_BY Policy
Warehouse HAS_STORAGE_PROFILE StorageProfile
Principal CAN_PLAN ScanPlan
Commit DERIVED_FROM Snapshot
LineageRun USED_BY QueryGraphModel
```

High-cardinality file and manifest facts should stay queryable through Iceberg/Sail metadata-as-data unless Grust provides a reusable taxonomy and storage strategy for them. The graph should be powerful, but the catalog must not smuggle a second lakehouse engine into its event sink.

The current local direction already proves the boundary: LakeCat's `grust-local` sink calls Grust-owned LakeCat projection helpers, and the Grust Cypher boundary test verifies catalog graph projection without making LakeCat own Cypher parsing, traversal, or graph execution. The current boundary test writes table-adjacent `Column`, `Snapshot`, and `Commit` events plus `Principal`, `ScanPlan`, tenant-root `Server`, and credential-root `StorageProfile` catalog events through Grust, then matches catalog-event labels through Grust Cypher. Storage profile replay uses redacted evidence such as `secret-ref-present` and the secret-reference provider, never the full secret-store URI. Credential-vend attempts replay through that same thin boundary as `StorageProfile`

graph events keyed by the redacted credential-root anchor, so QueryGraph can see a principal attempted credential-root access without seeing a secret reference or raw credential material. That proves QueryGraph can discover the semantic anchors LakeCat emits while the richer node/edge materialization remains reusable Grust work.

1.17 TypeSec For Security

LakeCat is a policy enforcement point, not the author of security semantics. TypeSec owns the semantics: RBAC, ODRL-style policy composition, typed capabilities, TypeDID envelopes, secure agents, credential issuance decisions, and authorization proofs.

Every externally meaningful action should pass through TypeSec:

- catalog configuration reads;
- namespace creation and listing;
- table creation, load, scan planning, commit, drop, and restore;
- credential vending;
- policy management;
- graph and lineage reads.

LakeCat gathers the request context and asks TypeSec for a decision. The context can include principal DID, agent DID, bearer-derived subject, warehouse, namespace, table, columns, snapshot, requested credential duration, purpose, and active policy bindings. TypeSec returns a decision and receipt. LakeCat persists the receipt with audit-safe hashes and applies the resulting restrictions before Sail plans.

This is where ODRL becomes operational. An ODRL-style policy can say that a principal may read only certain columns, only for a purpose, or only with an enforced row predicate. LakeCat parses the minimal enforceable subset it needs to narrow the plan, but policy composition and authorization semantics belong to TypeSec. LakeCat should ask TypeSec, not grow a parallel security language. When that subset is expressed through ODRL constraints, LakeCat accepts only operators that actually mean "use this as the allowed or narrowing value"; missing or deny-shaped operators fail closed instead of being treated as governed read permission. The bounded parser accepts camel-case, kebab-case, and prefixed JSON-LD operand keys such as `odrl:leftOperand` and `odrl:rightOperand`. It also accepts compact JSON-LD term objects such as `{"@id":"odrl:eq"}` for constraint operands and operators, plus JSON-LD `@value` and `@list` right operands for bounded allowed-column, purpose, and credential-TTL values. Malformed JSON-LD lists still fail closed, and the parser does not turn LakeCat into a full ODRL reasoner. Recognized constraint operands must also include a right operand; otherwise LakeCat rejects the policy material instead of silently dropping an allowed-column, row-predicate, purpose, or credential-TTL restriction. The derived restriction also rejects empty or blank allowed-column lists and blank purposes before they can reach credential issuance or governed Sail planning and fetch paths. The service route pins this behavior too: a table scan with a malformed active ODRL restriction, including malformed JSON-LD allowed-column lists, fails before Sail planning and before `table.scan-planned` replay evidence is emitted. A `fetchScanTasks` call with the same malformed JSON-LD active policy fails before Sail fetch execution and before `table.scan-`

`tasks-fetched` replay evidence is emitted, and a credential request with the same malformed JSON-LD active policy fails before issuer dispatch and before `credentials.vend-attempted` replay evidence is emitted. Purpose is composed the same way: every purpose source in the active policy material must agree. If one binding says a read is for `resilience-demo` and another says `training`, LakeCat rejects the restriction instead of guessing which purpose should follow the agent into Sail planning, credential TTL proof, and QueryGraph handoff evidence.

Credential vending follows the same rule. Raw credential vending is an audited exception. Governed Sail-planned reads are the default path for agents and untrusted principals. When credentials must be issued, TypeSec checks the `credentials.issue` capability for the exact secret reference and LakeCat returns only scoped, short-lived credential configuration. If policy carries a `max-credential-ttl-seconds` restriction, LakeCat passes that cap to the credential issuer and annotates each returned credential with `lakecat.max-credential-ttl-seconds`, so the exception path has an auditable duration bound. If the cap appears in multiple supported ODRL locations in the same policy document, or across multiple active policy bindings, LakeCat keeps the tightest value before asking the credential issuer. If an issuer returns that LakeCat TTL key itself, LakeCat normalizes the response to one TTL entry per credential and keeps the stricter valid TTL, so duplicate backend-supplied entries cannot widen or confuse the policy cap. The same response boundary owns the rest of the LakeCat evidence: issuer-supplied values for `lakecat.storage-profile-id`, `lakecat.storage-provider`, `lakecat.credential-mode`, `lakecat.authorization-principal`, `lakecat.governed-read-required`, and `lakecat.secret-ref-provider` are removed and replaced with catalog-derived values before the response is returned. The REST credential-vending regressions exercise this at the public response boundary: a backend can return multiple TTL entries or forged catalog evidence, but `loadCredentials` exposes one canonical proof while preserving issuer-owned credential details such as credential kind and provider session tokens. LakeCat records the same decision shape in audit/outbox evidence without copying raw credentials: each vended credential gets a hashed prefix, canonical LakeCat evidence values, and a hash of issuer-owned config. Replay can prove the response posture, but it does not inherit cloud session tokens. If the credential event carries a governed read restriction, outbox admission requires the top-level `read-restriction` to match `authorization-receipt.context.read-restriction`, keeping TTL and blocked-read evidence inside the durable receipt. Raw credential exceptions follow the same rule: the top-level `lakecat:raw-credential-exception` object must match `authorization-receipt.context.lakecat:raw-credential-exception` exactly, so trusted-human exceptions and blocked-agent denials cannot drift during replay.

1.18 Rust-First Engines And The V3 To V4 Path

The new Rust-first engine path matters because it changes where catalog intelligence can live. Sail is not a Java service with Rust bindings bolted on the side. It is a Rust engine path built around Arrow, DataFusion, generated Iceberg REST models, catalog provider traits, manifest pruning, metadata-as-data scans, and table-status conversion.

That shape gives LakeCat a better option than reimplementation. LakeCat can ask Sail for typed Iceberg behavior instead of parsing just enough JSON to survive a request. That matters for

Iceberg v3 and the emerging v4 work. Format v3 already pushes catalogs and engines toward richer metadata, row lineage, deletion semantics, and better interoperability around advanced table state. Format v4 is still settling, but it is plainly moving the lakehouse toward more adaptive and structured metadata rather than less.

LakeCat should evolve under three rules:

1. Conform to Iceberg v3 for ordinary clients.
2. Preserve unknown and emerging v4 metadata without claiming settled semantics.
3. Prefer typed Sail support as soon as Sail exposes it, using JSON passthrough only as a compatibility bridge.

That gives LakeCat room to support v4-ready capability flags, round-trip tests, metadata extension preservation, and future metadata-tree planning without forking Iceberg. The catalog can become more intelligent while the table remains portable. The standard path stays boring. The governed Sail path becomes richer.

The current v4 bridge is intentionally narrow and tested as such. When LakeCat sees `format-version: 4`, it does not pretend that Sail already has a settled typed v4 model. Instead, `lakecat-sail` extracts the stable JSON envelope fields that remain useful across versions: table UUID, location, schema id, snapshot id, sequence number, manifest-list path, default spec, and field names. It can plan a governed manifest-list scan task from that envelope and validate the signed plan task again during `fetchScanTasks` so a stateless fetch cannot drift to a different manifest list or widen the governed projection and filters. It also validates stable commit requirements such as table UUID, current schema id, main snapshot id, last assigned field id, and default spec id. Pruning and typed metadata-tree semantics wait for Sail-owned v4 support.

1.19 OSI, OpenLineage, And Responsible Semantic Handoff

QueryGraph needs more than physical table access. It needs a semantic picture: datasets, fields, policies, lineage, graph relationships, and anchors that can survive import. LakeCat should publish that picture without pretending to be QueryGraph.

The LakeCat bootstrap bundle contains:

- Semantic Croissant and CDIF projections for dataset and field discovery.
- An OSI handoff with stable dataset and field anchors.
- ODRL policy artifacts and TypeSec policy context.
- OpenLineage events for catalog changes, scan plans, commits, and maintenance.
- A Grust-ready graph envelope.
- A manifest that hashes each emitted artifact.

The OSI boundary is deliberately careful. LakeCat should not author rich business metrics, dimensions, joins, ontology claims, or authoritative semantic names. It can publish stable anchors and governed source metadata. QueryGraph owns the final semantic model.

This is the Responsible Semantic Layer boundary. Semantic Croissant and CDIF make datasets and fields discoverable and exchangeable. OSI gives QueryGraph a stable handoff for semantic

anchors without forcing LakeCat to own business meaning. OpenLineage records how catalog and planning events happened. Together they let the semantic layer be responsible because it can be traced back to catalog state, policy, and lineage, not just to a hand-authored model file.

OpenLineage fits the catalog outbox. A committed table create, scan plan, commit, soft delete, restore, or maintenance action can become a lineage event. Because the event is drained from a durable outbox after the catalog transaction, lineage reflects committed state rather than a handler's best-effort side effect. Drains process pending events in `created_at,event_id` order before projection, response summarization, and delivery acknowledgement. That stable order is part of the replay contract QueryGraph and OpenLineage consume, and it holds even when a custom store implementation returns a pending batch in another order.

1.20 QueryGraph.ai When LakeCat Is Done

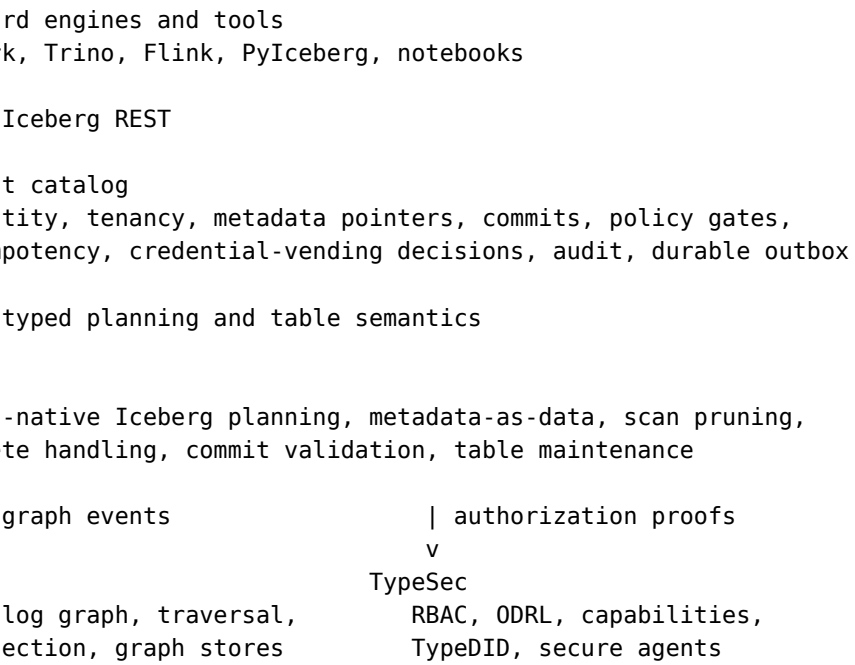
QueryGraph.ai is the enterprise lakehouse this work is pointing toward. QueryGraph needs the catalog to be more than a storage address book. It wants to answer questions over data, metadata, semantics, policy, and lineage as one governed graph. That requires a catalog foundation that can speak to ordinary Iceberg clients and also publish trustworthy control-plane facts.

LakeCat supports that foundation by exposing a QueryGraph bootstrap endpoint:

```
/querygraph/v1/bootstrap
```

The bundle gives QueryGraph an import contract. QueryGraph can verify hashes, load catalog graph envelopes through Grust, inspect policy artifacts through TypeSec, and attach semantic modeling work to stable dataset and field anchors. The import path should prove that LakeCat is the substrate, not a standalone demo.

When LakeCat is done, the QueryGraph.ai architecture looks like this:



```
    |
    | semantic and lineage bootstrap
    v
QueryGraph.ai
  Responsible Semantic Layer over Croissant, CDIF, OSI,
  OpenLineage, ODRL, graph, and governed table access
```

Basic catalog use remains optional and standard. A normal Iceberg engine can load and commit tables without knowing QueryGraph exists. The enhanced path is there for enterprises that want more: governed Sail-planned reads, TypeSec authorization receipts, ODRL rights, TypeDID agent identity, OpenLineage replay, Semantic Croissant/CDIF publication, OSI handoff, and Grust graph loading.

That is the core motivation for LakeCat. The next QueryGraph.ai should not bolt governance, graph, and lineage onto tables after the fact. It should begin from a catalog that already records governed state transitions and exposes standard, engine-close planning.

1.21 Implementation Shape

The current workspace shape expresses the architecture directly:

```
crates/
  lakecat-core        stable IDs, errors, time, config, content hashes
  lakecat-api         Iceberg REST request/response adapters
  lakecat-store       catalog state traits and Turso-backed implementation
  lakecat-sail        Sail provider bridge and privileged planning client
  lakecat-graph       catalog-facing Grust sink/adapters
  lakecat-security    TypeSec integration and authorization receipts
  lakecat-lineage     OpenLineage projection and event receipts
  lakecat-querygraph  Croissant/CDIF/OSI/ODRL/OpenLineage bootstrap projection
  lakecat-service     axum service, middleware, auth, routing
  lakecat-cli         admin, local demo, conformance, bootstrap export
```

Feature gates keep integrations honest:

```
sail-local    use local Sail APIs for planning and provider integration
typesec-local use local TypeSec APIs for governance and TypeDID verification
grust-local   use local Grust APIs for catalog graph projection
turso-local   use the Turso-backed durable store
```

Embedded defaults stay safe for tests. Real integrations are explicit. That matters because LakeCat is a foundation, not a pile of optional demos. A test that only uses memory stores should not accidentally depend on a sibling repo. A test that claims to validate TypeSec should enable `typesec-local` and call TypeSec.

The dependency contract is executable because LakeCat still has one active sibling bridge. Grust and TypeSec now resolve from the published `grust-graph` 0.9.0, `grust-cypher` 0.9.0, and `typesec` 0.8.0 crates, so the `grust-local` and `typesec-local` features no longer require sibling checkouts merely to compile. That makes the graph and governance boundaries reproducible outside this machine while still keeping their reusable behavior in Grust and TypeSec.

Sail is different today: LakeCat still uses local Sail paths plus a checked-in patch bridge for helper APIs that are not yet published. Before pushing a slice that touches integration features, run:

```
scripts/check-local-dependency-contract.sh
```

The script checks the manual-only CI trigger, scans every GitHub workflow file for forbidden automatic cloud triggers, verifies crates.io resolution for the published Grust graph/Cypher and TypeSec versions, the local Sail path bridge, the Sail patch files manual CI applies, and the concrete Sail helper API surface LakeCat uses: generated Iceberg REST models, typed metadata inputs, planning result helpers, fetchScanTasks result helpers, and table-status conversion. It also checks the local QueryGraph Rust importer for the LakeCat view receipt-chain contract: `receipt-chain-hash` must be preserved in view receipt evidence and missing receipt-chain evidence must fail closed. Manual-only means no automatic push, pull-request, pull-request-target, merge-queue, repository-dispatch, scheduled, workflow-run, or reusable-workflow cloud runs; the local audit fails if any of those triggers appear before the local gates are proven stable, including compact scalar triggers, inline lists and maps, block lists and maps, quoted YAML forms such as `"on": ["push"]`, and quoted event names in trigger blocks. The guard looks specifically at the workflow `on:` declaration, so a harmless job id such as `jobs.push` is allowed. The focused workflow-trigger self-test exists so this guard can be checked without running the full dependency audit. It is not a substitute for upstreaming the Sail helper APIs or re-enabling automatic CI; it is a guard that makes drift visible while LakeCat still depends on unpublished Sail helper work and a local QueryGraph acceptance target.

For the first release, LakeCat has one local release gate:

```
scripts/check-release-readiness.sh
```

The full gate runs the dependency contract, the workspace formatting matrix, default workspace tests, QGLake fixture coverage, Turso store tests, Sail, TypeSec, and Grust integration feature tests, an explicit all-features CLI test, the all-features workspace library test, the book build, and the QGLake handoff proof. The default workspace test still covers ordinary doc-tests; the feature matrix targets package unit tests so an empty rustdoc phase cannot hang after the actual Turso/Sail/TypeSec/Grust coverage has passed. The `--quick` mode keeps script syntax, dependency-contract, formatting, and diff checks cheap enough to run inside narrow implementation slices. Cloud CI remains manual-only until this local gate is boringly green.

As of the current local reconciliation, the Sail helper work is not an anonymous dirty tree. `/Users/alexy/src/sail` has scoped local commits on `codex/graph` for exposing Iceberg REST models to LakeCat, preserving Iceberg manifest lower and upper bounds in Avro, and adding Sail's Cypher graph query extension. That Cypher extension is a Sail SQL/analyzer/planning surface; the catalog graph taxonomy, projection helpers, traversal, and stores remain Grust responsibilities. The only remaining Sail working-tree entries are untracked artifact/book directories, and pushing the Sail branch upstream is blocked by HTTPS GitHub authentication rather than by local test failures.

1.22 Standard Compatibility And Extensions

LakeCat must be boring where standards require boring behavior. Standard Iceberg clients should be able to use the catalog without knowing QueryGraph exists. Business semantics and agent state must not be required custom Iceberg metadata.

Extensions belong beside the standard path:

- `/catalog/v1` serves Iceberg REST compatibility.
- management APIs handle warehouses, policies, profiles, and operational state.
- `/querygraph/v1/bootstrap` publishes QueryGraph import artifacts.
- feature-gated Sail paths provide governed scan planning and local provider integration.

Format v4 work should follow the same rule. Prefer typed Sail support when available. JSON passthrough can bridge compatibility, but it is not the desired long-term implementation. Round-trip tests should prove LakeCat preserves unknown or evolving metadata without claiming settled semantics too early.

1.23 Workflow Examples

The catalog is easiest to understand by watching it participate in ordinary work. LakeCat should not ask users to think about graph, lineage, security, and Sail every time they read a table. Those systems should appear when they matter: at the boundary where a name is resolved, a policy is enforced, a plan is created, credentials are withheld or issued, and a durable event is replayed.

The examples below use one table, `local.default.events`, but the pattern is the same for larger warehouses. The important point is not the exact sample data. It is the catalog role in each workflow.

1.23.1 Starting The Catalog

A local operator starts LakeCat as an Iceberg REST catalog plus management surface:

```
cargo run -p lakecat-service --features sail-local,turso-local,typesec-
local,grust-local
```

The standard catalog path is still `/catalog/v1`. The management and QueryGraph surfaces sit beside it:

```
/catalog/v1
/management/v1
/querygraph/v1/bootstrap
```

A simple health-oriented configuration read shows the split. Standard engines care about the Iceberg endpoints. Operators and QueryGraph care about the management and bootstrap endpoints.

```
curl -s http://127.0.0.1:3000/catalog/v1/config
```

The defaults intentionally separate compatibility from future capability:

```
{
  "defaults": [
    {"key": "lakecat.compatibility", "value": "iceberg-rest"},
```

```
    {"key": "lakecat.format.baseline", "value": "iceberg-v1-v3"},
    {"key": "lakecat.format.v4", "value": "extension-ready"},
    {"key": "lakecat.format.v4.bridge", "value": "json-passthrough"},
    {"key": "lakecat.format.v4.typed-sail", "value": "unavailable"}
  ]
}
```

That means LakeCat can preserve and replay emerging v4 metadata through the Sail JSON bridge, but it is not claiming typed Sail v4 semantics yet. The same defaults are stored in catalog config-read replay evidence, and malformed replay that omits the v4 bridge posture is rejected before graph or OpenLineage projection. The replay defaults must also be ordinary string key/value entries with duplicate-free keys, so a saved outbox event cannot say both `lakecat.format.v4.typed-sail=unavailable` and `lakecat.format.v4.typed-sail=available`.

The bridge is intentionally conservative, but it should not reject Iceberg metadata that Sail has already decoded. Manifest expansion now emits null partition slots as JSON `null` and recursively encodes nested Sail partition literals into JSON objects, arrays, and explicit key/value map entries. That keeps standard Iceberg REST fetch responses usable for richer partition tuples without pretending LakeCat owns a full typed v4 implementation.

At this point the catalog is already doing more than route HTTP. It has a warehouse identity, a store, a governance engine, a Sail planning seam, a graph sink, and a lineage sink. Embedded defaults keep the local loop small, but the same trait boundaries can point to Turso, TypeSec, Grust, and Sail.

1.23.2 Registering The Warehouse Shape

An operator usually starts with management objects. A server groups projects. A project groups warehouses. A warehouse owns namespaces, tables, views, storage profiles, policy bindings, and the metadata pointer state that standard engines see through Iceberg REST.

```
curl -s -X PUT http://127.0.0.1:3000/management/v1/servers/prod \
  -H 'content-type: application/json' \
  -d '{
    "display-name": "Production LakeCat",
    "endpoint-url": "https://lakecat.example.com",
    "properties": {
      "owner": "platform"
    }
  }'

curl -s -X PUT http://127.0.0.1:3000/management/v1/projects/resilience \
  -H 'content-type: application/json' \
  -d '{
    "display-name": "Resilience Desk",
    "server-id": "prod",
    "properties": {
      "environment": "demo"
```

```
    }
  }'

curl -s -X PUT http://127.0.0.1:3000/management/v1/projects/resilience/
warehouses/local \
  -H 'content-type: application/json' \
  -d '{
    "display-name": "Local QGLake Warehouse",
    "storage-root": "file:///tmp/lakecat/qglake",
    "properties": {
      "querygraph": "enabled"
    }
  }'
```

These writes are not Iceberg table metadata. They are catalog control-plane state. LakeCat persists them durably, records authorization receipts, and writes outbox events. When the outbox drains, server, project, warehouse, and storage-profile and policy-binding changes become catalog graph events; the same management changes also become OpenLineage receipts. QueryGraph can later learn the management shape without requiring every Iceberg client to understand it. Project, server, and warehouse tenant-root replay is checked before projection: project evidence must carry a matching project id, optional valid server scope, and string-map public properties; server evidence must carry a valid server id, optional valid endpoint URL or full `endpoint-url-hash`, and string-map properties; warehouse evidence must carry a valid warehouse, project id, optional valid storage root or full `storage-root-hash`, and string-map properties. Policy-binding upsert replay is checked before projection too: the evidence must carry a valid policy id, warehouse, optional namespace/table scope, an enforcement flag, and the captured ODRL material. LakeCat does not reason over that ODRL during replay, but malformed binding shape fails closed before the policy anchor can be delivered to graph or lineage sinks. Those management upserts must also carry a valid authorization receipt principal, so the catalog graph and OpenLineage stream never accept actorless tenant-root, storage-profile, or policy mutations. Namespace lifecycle replay is checked before projection as well: create, load, and drop events must carry a valid warehouse and either a valid namespace path or non-empty namespace component array. A malformed namespace lifecycle event stays pending and reaches neither the Grust-facing graph sink nor OpenLineage. Catalog read replay has the same fail-closed shape: `catalog.config-read` events must carry a valid warehouse, and `namespace.listed` events must carry both a valid warehouse and an unsigned namespace count before the read evidence can be projected. These standard catalog reads and namespace lifecycle events must also carry a valid authorization receipt principal before delivery, so Iceberg-compatible control-plane replay remains attributable. Management-list replay is checked before delivery too: policy-binding, project, server, storage-profile, and warehouse list events must carry unsigned counts, warehouse-scoped lists must carry a valid warehouse, and optional project scope must be a string before those reads can become replay evidence. View replay is checked at the same boundary: view list events must carry valid warehouse, namespace, and count evidence, while view create/load/drop evidence must carry a valid warehouse, namespace, and non-empty view name before graph or OpenLineage projection. View list and lifecycle replay must also carry a valid authorization receipt principal

before delivery, preserving actor evidence for QueryGraph view proofs. Table lifecycle replay now follows the same rule: create, load, delete, and restore events must carry a valid root table identity, and any payload warehouse, namespace, table-name, or soft-delete table evidence must agree with that identity before the event can be acknowledged. Server endpoint URLs are operator-visible management metadata, so LakeCat keeps them deliberately plain: they must be absolute `http` or `https` URLs, and they cannot include query strings, fragments, or URI userinfo. Rejected submissions return `server-endpoint-url-hash=sha256:...` evidence rather than echoing the submitted endpoint. Warehouse replay does not forward the raw storage root to graph or lineage consumers. The drained payload replaces `storage-root` with `storage-root-hash`, so QueryGraph can bind tenant evidence to a configured root without receiving the local filesystem path or bucket URI.

1.23.3 Storage Profiles And Credential Roots

Storage profiles bind a warehouse to physical storage roots and credential issuance policy. A local profile can return scoped local file configuration. A remote profile should usually reference a secret store and require TypeSec to authorize issuance before any resolver sees the secret reference. Warehouse storage roots are validated before memory or Turso persistence: query strings, fragments, URI userinfo, and literal or percent-encoded dot path segments fail with `warehouse-storage-root-hash=sha256:...` evidence rather than echoing the submitted root. LakeCat rejects profiles whose declared provider conflicts with the URI scheme of the location prefix, so a credential root cannot claim to be local while pointing at an S3 prefix. Those provider/location mismatch errors follow the same redaction rule as replay: they name provider labels and a `storage-profile-prefix-hash=sha256:...`, not the raw storage root. When multiple profiles in the same warehouse could match a table, LakeCat uses the longest matching location prefix. If two profiles tie on that longest prefix, LakeCat fails closed rather than guessing which credential root or metadata-object boundary should apply. The ambiguity error reports the competing profile ids and `location-prefix-hash=sha256:...` evidence, not the raw storage root. The location prefix itself must be plainly addressed: LakeCat rejects literal and percent-encoded dot path segments, query strings, fragments, and URI userinfo before the profile can reach memory or Turso persistence. Traversal-shaped or decorated storage-profile prefixes fail with `storage-profile-prefix-hash=sha256:...` evidence rather than echoing the raw prefix, token-like query value, or embedded userinfo. The management route pins the same operator-facing behavior, so a rejected storage-profile upsert does not leak the submitted decorated prefix. It also rejects unsafe issuance-mode combinations: `local-file-no-secret` is for file storage only, while `short-lived-secret-ref` is for configured remote providers such as S3, GCS, and Azure. Those mismatches fail with the same `storage-profile-prefix-hash=sha256:...` anchor and without echoing the raw storage prefix or submitted `secret-ref`, so operators can correlate the credential-root error without turning the management API into a credential leak. The `public-config` map is only for non-secret routing hints such as region, endpoint labels, and operational purpose. LakeCat rejects secret-looking public keys and values, so raw tokens, passwords, access keys, and credential query parameters must move behind `secret-ref` and the TypeSec-authorized resolver path. That rule is enforced both when a profile is built from a management request and when a storage profile is revalidated before memory or Turso persistence, so deserialized control-

plane records cannot bypass the public-config guard. Public-config validation failures also use `public-config-key-hash=sha256:...` evidence rather than echoing the submitted key or value, because even a rejected key name may contain a secret-looking identifier. LakeCat also reserves credential-evidence keys such as `lakecat.storage-profile-id`, `lakecat.storage-provider`, `lakecat.credential-mode`, and `lakecat.max-credential-ttl-seconds`; operators may still publish non-secret hints such as `lakecat.endpoint`, but they cannot shadow catalog-owned proof in the eventual credential response. The `secret-ref` field itself must remain a clean external secret-store locator: LakeCat rejects query strings, URI fragments, and userinfo before persisting a storage profile, so token-like material cannot hide inside a decorated secret URI. It also rejects literal and percent-encoded dot path segments, so a credential root cannot rely on traversal-like spelling before a resolver sees it. Unsupported credential-root schemes and malformed secret-root paths are rejected with `secret-ref-hash=sha256:...` evidence instead of echoing the submitted secret reference. The same hash-only rule applies to invalid secret-ref URI syntax, decorated URI forms, and embedded secret-like material such as password or token assignments. Management upsert and list responses follow the same redaction rule. They do not echo the raw `secret-ref`; they return `secret-ref-present`, `secret-ref-provider`, and `secret-ref-hash` so operators and QueryGraph can verify that a credential root exists and correlate it without learning the secret-store path. When LakeCat selects the storage profile for a table, the location prefix is also matched on a storage-root boundary. A profile for `s3://lakecat/events` applies to that exact root and to children such as `s3://lakecat/events/tenant-a/table`, but it does not apply to a sibling path such as `s3://lakecat/events-shadow/table`. That keeps credential roots from accidentally governing more storage than their configured prefix describes; if no stored profile matches, LakeCat falls back to an inferred governed-read profile for the table location. The same check runs after the credential issuer returns: `loadCredentials` rejects any returned prefix broader than the selected profile before LakeCat attaches canonical response evidence, so a custom issuer cannot widen catalog-owned storage scope. The rejection exposes only `credential-prefix-hash` and `storage-profile-prefix-hash` evidence, and LakeCat records no `credentials.vend-attempted` replay event for that failed issuer response.

```
curl -s -X PUT \
  http://127.0.0.1:3000/management/v1/warehouses/local/storage-profiles/local-
events \
  -H 'content-type: application/json' \
  -d '{
    "location-prefix": "file:///tmp/lakecat/qglake/events",
    "provider": "file",
    "issuance-mode": "local-file-no-secret",
    "public-config": {
      "lakecat.purpose": "developer-loop"
    }
  }'

curl -s -X PUT \
  http://127.0.0.1:3000/management/v1/warehouses/local/storage-profiles/s3-events
\
```

```
-H 'content-type: application/json' \
-d '{
  "location-prefix": "s3://lakecat/events",
  "provider": "s3",
  "issuance-mode": "short-lived-secret-ref",
  "secret-ref": "vault://kv/lakecat/events",
  "public-config": {
    "lakecat.region": "us-west-2",
    "lakecat.purpose": "production-events"
  }
}'
```

The catalog row stores the public profile and secret reference, not raw cloud keys. A later credential request is checked against TypeSec and against the effective read restriction for the target table. Agents with fine-grained table restrictions are steered to governed Sail-planned reads instead of raw credentials. Trusted humans can receive audited standard credentials only when policy allows the exception. For production secret managers, LakeCat keeps a provider-dispatch seam rather than hard-coding credentials into catalog state. `vault://` can resolve through the built-in Vault HTTP backend when Vault environment configuration is present. `aws-sm://`, `gcp-sm://`, and `azure-kv://` can dispatch to explicitly configured provider backends after TypeSec authorizes the exact secret-ref resource. If no backend is configured, those providers fail closed with an operator-readable not-configured error, and denied TypeSec decisions do not call the backend at all. Configured provider backends receive the same policy-derived `max-credential-ttl-seconds` cap that LakeCat records in the read restriction, and returned credentials must preserve that cap in `lakecat.max-credential-ttl-seconds`. LakeCat rewrites duplicate TTL config entries into one effective value before returning credentials, preserving a stricter issuer TTL when it is valid and otherwise falling back to the policy cap. It also rewrites LakeCat-owned profile, provider, mode, principal, and governed-read-required evidence after issuance. For secret-ref-backed profiles it also derives `lakecat.secret-ref-provider` from the selected storage profile, so a cloud secret backend cannot make the response look like a different catalog decision or secret-provider path. Replay admission treats that evidence as structural too: secret-ref providers must be nonblank when `secret-ref-present` is true, and provider/hash fields must be absent when `secret-ref-present` is false, no matter how a corrupted pending event encodes them. The service tests for the REST credential endpoint prove this response shape directly, not just through helper functions. LakeCat also rejects any credential whose returned prefix is outside the storage profile's `location-prefix`, so a misconfigured cloud secret backend cannot widen a table's storage scope after TypeSec has authorized the secret reference. That failure remains hash-only and stops before credential-vend replay evidence is recorded. The audit event for the credential attempt records redacted `credential-response-evidence`: the response prefix is hashed, LakeCat-owned proof fields are kept as canonical values, and issuer-owned config is hashed rather than copied. That keeps OpenLineage and QueryGraph replay useful without turning lineage into a credential leak. For secret-ref-backed profiles the redacted response evidence includes the catalog-derived `lakecat.secret-ref-provider`, while the storage-profile replay evidence includes `secret-ref-provider` and a full `secret-ref-hash`; outbox admission rejects any credential response whose provider proof drifts from the selected profile before graph or OpenLineage projection. The

nested storage-profile proof is still checked even when no credentials are returned: provider and issuance mode must be compatible, and secret-reference presence must match the mode. That keeps blocked credential attempts from projecting a weaker credential-root proof than storage-profile management would accept. The storage-profile and credential-vend service tests pin that producer-side `location-prefix-hash` evidence is already a full SHA-256 digest before QGLake receives the compact `locationPrefixHash` proof. The trusted-human credential-vending route test pins that the committed outbox payload contains this redacted proof for the audited raw-credential exception path. The blocked-agent route pins the other side of the same contract: when Sail-planned reads are required and no raw credentials are returned, the outbox records an explicit empty `credential-response-evidence` array rather than leaving replay to infer why no credential proof exists. A not-configured resolver error reports the provider label and a `secret-ref-hash=sha256:...` value, not the raw secret URI, so the operator can correlate configuration without leaking the credential root. Resolver validation errors for malformed Vault and TypeSec environment references follow the same rule: wrong schemes, missing Vault mounts or paths, and invalid environment-variable names produce hash evidence instead of echoing the malformed secret reference. Generic provider detection and resolver URI parsing follow that rule too, including unsupported provider schemes, so malformed credential-root strings cannot leak through production resolver diagnostics. Once a configured resolver is authorized to run, backend lookup and secret payload parse failures still stay hash-only: LakeCat returns the secret-reference hash and an error-detail hash instead of the environment variable name, Vault path, token, namespace, backend exception text, or malformed secret fields. Secret payload parsing also rejects malformed credential configuration before issuance, including blank config keys in either object-shaped secrets, ConfigEntry-array secrets, or Vault's nested data object. That keeps secret-manager output from turning into ambiguous Iceberg client credential configuration. When storage-profile changes replay into lineage/OpenLineage evidence, LakeCat does not forward the full secret-store URI or raw storage root. The committed audit/outbox payload keeps `secret-ref-present` and `secret-ref-provider` so QueryGraph can verify that a production credential root exists without learning the Vault, cloud secret manager, or TypeSec environment path. It also records a full `location-prefix-hash` instead of raw `location-prefix`, so replayed evidence can bind a credential root to a storage scope without exposing the bucket, path, or local filesystem root to downstream consumers. Warehouse replay follows the same shape: `storage-root` becomes `storage-root-hash` before graph and lineage projection, keeping the tenant root replayable without exposing the raw root itself. The drain also rejects unsafe storage-profile upsert evidence before delivery: `storage-profile.upserted` must carry a full `location-prefix-hash` and must not carry raw `location-prefix` or raw `secret-ref` values. If secret-reference presence is true, the replay evidence must carry a provider and full `secret-ref-hash`; if presence is false, provider and hash evidence must be absent. The same admission check validates the credential-root identity before projection: profile id must be non-empty, the nested warehouse must be valid and match any top-level warehouse field, and provider plus issuance mode must use LakeCat's supported storage-profile vocabulary. Provider and issuance-mode compatibility is replay-checked as well: `local-file-no-secret` is only valid for the file provider, and `short-lived-secret-ref` is only valid for cloud object providers. Secret-reference presence must also agree with issuance mode: short-lived secret-ref profiles must carry redacted secret-reference proof, while governed and no-

secret profiles cannot carry secret-reference proof. The drain summary lifts the same proof into compact fields alongside the profile id and provider. QGLake replay verification requires that compact storage-profile upsert evidence, which means a saved handoff can prove the credential root was configured without handing the next system a secret path.

1.23.4 A PySpark User Reads Iceberg

A PySpark user should not need to know about QueryGraph. They configure Spark's Iceberg REST catalog and point it at LakeCat:

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("lakecat-events")
    .config("spark.sql.catalog.lakecat", "org.apache.iceberg.spark.SparkCatalog")
    .config("spark.sql.catalog.lakecat.type", "rest")
    .config("spark.sql.catalog.lakecat.uri", "http://127.0.0.1:3000/catalog/v1")
    .config("spark.sql.defaultCatalog", "lakecat")
    .getOrCreate()
)

events = spark.table("default.events")
events.select("event_id", "severity").where("severity = 'critical'").show()
```

For an unrestricted principal, the flow looks like a normal Iceberg read:

1. Spark asks LakeCat to resolve `default.events`.
2. LakeCat checks the principal and table capability.
3. LakeCat returns an Iceberg-compatible table response.
4. Spark plans the read using Iceberg metadata.

For a governed principal, the more interesting path is server-side planning. The user request still looks ordinary, but LakeCat derives the mandatory restriction before Sail sees the plan. If policy allows only `event_id` and `severity`, then a wider client projection is narrowed:

```
client asks:     event_id, severity, raw_payload
policy allows:   event_id, severity
Sail receives:   event_id, severity
```

The catalog does not trust the client to remember that. The restriction is re-applied when scan tasks are fetched, and the audit payload records the policy hash, narrowed columns, row predicate, storage location, metadata location, principal, requested stats fields, and effective stats fields. The fetch response also carries LakeCat extension evidence for the exact `required-projection` and `required-filters` derived from the authorized capability. That makes a stateless `fetchScanTasks` replay prove the restriction was re-applied, not merely that the original policy object was echoed. The QGLake fixture verifier checks those fields directly when it fetches scan tasks, so a local acceptance run fails if the response drops either the narrowed projection or the mandatory row predicate proof. The same verifier now checks that the exported

policy binding, scan planning extension, and fetch extension all preserve the server-derived purpose and policy-derived `max-credential-ttl-seconds` cap before compact replay proof can be accepted. The scan-planned replay proof also carries `plannedRequestedStatsFields` and `plannedEffectiveStatsFields`, so QGLake can prove a broader stats request, the server-derived narrowing, and the final effective stats fields that match the allowed columns. Saved handoff summaries and captured replay output are rejected if those fields disappear, widen, or drift apart. It also cross-checks the embedded ODRL policy projection against the structured binding so QueryGraph cannot import a stale copy while LakeCat verifies a different one.

1.23.5 A Notebook Requests Credentials

Credential vending is deliberately different from scan planning. Returning storage credentials gives the client broader power than returning a governed task list, so LakeCat treats it as an exception path.

```
curl -s \
  -H 'x-lakecat-principal: agent:triage' \
  http://127.0.0.1:3000/catalog/v1/local/namespaces/default/tables/events/
credentials
```

For an agent bound by a fine-grained restriction, LakeCat should fail closed:

```
{
  "credentials": [],
  "lakecat:credential-block-reason": "fine-grained read restriction requires
Sail-planned reads"
}
```

That empty credential response is not a missing feature. It is the intended agentic posture. The agent should ask LakeCat to plan the read through Sail, not receive raw storage reach. The audit event records the decision and the lineage outbox can replay the credential-vend attempt with the same block reason.

For a trusted human principal, policy can allow an audited raw credential exception. LakeCat still records the same context:

```
principal: analyst:maya
principal-kind: human
table: local.default.events
decision: raw credential exception allowed
reason: trusted human principal
restriction: present in receipt
lineage: credential vend attempted
```

The contrast matters for operators. They can prove that agents were kept on the governed path while a human exception was explicit, policy-backed, and replayable.

1.23.6 A View Becomes Part Of The Catalog Story

Views are catalog objects too. LakeCat stores durable view records with SQL, dialect, schema version, typed columns, properties, creator, and warehouse scope. They can be managed through the management API or through warehouse-prefixed catalog routes.

```
curl -s -X PUT \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/views/
events_view \
  -H 'content-type: application/json' \
  -d '{
    "sql": "select event_id, severity from default.events where severity =
'\''critical'\''",
    "dialect": "spark-sql",
    "schema-version": 1,
    "columns": [
      {
        "name": "event_id",
        "data-type": {"type": "long"},
        "nullable": false,
        "comment": "Stable event identifier"
      },
      {
        "name": "severity",
        "data-type": {"type": "string"},
        "nullable": false,
        "comment": "Operational severity"
      }
    ],
    "properties": {
      "owner": "resilience-desk"
    }
  }'
```

This is not Iceberg business metadata glued onto a table. It is catalog state about a view object. LakeCat records `view.listed`, `view.upserted`, `view.loaded`, and `view.dropped` audit events. Outbox replay projects listing reads to namespace-scoped graph and OpenLineage evidence, and projects single-view changes and reads to catalog-facing View graph events plus LakeCat OpenLineage view dataset receipts. QueryGraph bootstrap can then include views with OSI hashes, store-assigned view versions, view-aware graph edges, and OpenLineage view counts. The lineage-drain summary also carries compact view replay identity:

```
{
  "name": "events_view",
  "view-version": 1,
  "schema-version": 1,
  "dialect": "sql"
}
```

That `view-version` is assigned by the durable store on each upsert, not by the caller. It is the first compatibility bridge toward Iceberg view commit history: QueryGraph can compare a bootstrap view artifact with the catalog's current view version today. A caller that wants optimistic commit behavior can include `expected-view-version` on the next upsert:

```
curl -s -X PUT \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/views/
events_view \
  -H 'content-type: application/json' \
  -d '{
    "sql": "select event_id from default.events where severity =
'\''critical'\''",
    "dialect": "spark-sql",
    "schema-version": 2,
    "expected-view-version": 1
  }'
```

If another writer has already advanced the view, LakeCat returns a conflict before it replaces the current view or appends a receipt. Omitting the field keeps the compatibility behavior: the store assigns the next version. The guard must be a positive store-assigned version. `expected-view-version=0` is rejected as a bad request before LakeCat changes the active view or appends any view-version receipt. The same guard can protect deletion:

```
curl -s -X DELETE \
  'http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/views/
events_view?expected-view-version=2'
```

If the current view is no longer version 2, LakeCat returns a conflict before it removes the view or appends a tombstone receipt. Accepted guarded mutations also carry their `expected-view-version` into the audit/outbox payload. During lineage drain, LakeCat turns that into compact replay evidence, so QueryGraph can distinguish "the replacement happened at version 2" from "the replacement was guarded by version 1 and then produced version 2."

LakeCat also writes a compact view-version receipt in the durable store. The receipt records the stable view id, assigned version, previous version, previous receipt hash, content hash, principal, operation, and timestamp. That makes the compact receipt list a hash chain: version 2 points at the version 1 receipt hash, and a later tombstone points at the last upsert receipt hash. Fuller version-log semantics remain a Sail-aligned implementation target. When a view is dropped, LakeCat appends a compact tombstone receipt instead of inventing a new view version: the receipt keeps `view-version` at the last durable version, sets `operation` to `drop`, links to the previous receipt, and preserves the last content hash so QueryGraph or an operator can prove which catalog view state was removed. If the same view name is later recreated, the new upsert continues after the latest receipt in that durable chain. A create, drop, and recreate sequence therefore looks like version 1 upsert, version 1 drop tombstone, version 2 upsert linked to the tombstone receipt, not two unrelated version-1 chains for the same stable view id.

```
{
  "event-type": "view.upserted",
```

```
  "view-warehouse": "local",
  "view-namespace": ["default"],
  "view-name": "events_view",
  "view-stable-id": "lakecat:view:local:default:events_view",
  "view-version": 2,
  "expected-view-version": 1,
  "graph-events": 2,
  "lineage-events": 1,
  "replay-event-hashes": ["sha256:..."],
  "replay-open-lineage-hashes": ["sha256:..."]
}
```

A `view.listed` replay is intentionally namespace-scoped. It records the warehouse, namespace, view count, graph and lineage projection counts, and receipt hashes for the list read without fabricating a single `view-stable-id`.

QGLake acceptance compares both that `view-stable-id` and `view-version` with the accepted QueryGraph bootstrap view artifacts. That closes a small but important gap: the bootstrap bundle may say a view was exported, and the drain evidence can now prove the corresponding view catalog event was replayed at the same durable catalog version with graph and lineage receipts. When the event was guarded, QGLake replay JSON also includes `expectedViewVersion`, preserving the optimistic version that LakeCat checked before accepting the mutation. The live QGLake fixture uses that path for deletion: after QueryGraph accepts the transient view in the bootstrap bundle, the fixture drops it with `expected-view-version` equal to the accepted durable view version.

When QueryGraph bootstrap is replayed through the outbox, LakeCat includes only compact receipt hashes:

```
{
  "event-type": "querygraph.bootstrap",
  "view-artifact-count": 1,
  "view-version-receipt-hashes": ["sha256:..."]
}
```

QGLake acceptance requires one non-empty receipt hash for each accepted view artifact. That keeps normal Iceberg view access standard, but gives the QueryGraph handoff a durable proof that the exported view version came from LakeCat's catalog spine. Compact handoff verification also binds those bootstrap receipt hashes to `viewReceiptChainProof.views[].acceptedReceiptHash` exactly, so a saved summary cannot splice a valid-looking QueryGraph bootstrap receipt array from another accepted view proof. The fixture also exercises the deletion side of the same workflow: it creates a transient view, accepts a QueryGraph bootstrap that contains that view, drops the view, reads the receipt chain through the governed management endpoints by view name and namespace, and then requires lineage-drain replay to include `view.dropped`, `view.version-receipts-listed`, and `view.version-receipt-chains-listed` evidence with non-empty tombstone receipt hashes and namespace chain hashes. The service route tests pin those produced `receipt-hash`, `view-hash`, and `chain-hash` fields as full SHA-256 digest evidence before the

QGLake verifier consumes them. LakeCat also validates the ordered `previous-receipt-hash` links before marking a namespace chain as `chain-verified`, so QueryGraph can reject a replay that contains hashes but not a coherent chain.

The verifier is fail-closed on version progression too. The first receipt must be a version-1 upsert with no previous version or receipt hash; zero-version chains, first-receipt tombstones, and first receipts with forged previous-link fields fail before the chain can be marked verified. Later upserts must point at the previous receipt hash and advance exactly one durable view version. Drop tombstones must point at the previous receipt hash, cite the previous view version, and keep `view-version` equal to the accepted version that was deleted. Unsupported operations and forged `previous-receipt-hash` links fail the same check. That lets QueryGraph reject a chain that is cryptographically linked but lies about how the catalog view advanced.

QueryGraph and operators can also read the compact receipt chain directly from the governed management surface:

```
curl -s \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/views/
events_view/version-receipts \
  -H 'x-lakecat-agent-did: did:example:resilience-agent'
```


```
{
  "receipts": [
    {
      "stable-id": "lakecat:view:local:default:events_view",
      "view-version": 1,
      "previous-view-version": null,
      "operation": "upsert",
      "view-hash": "sha256:...",
      "receipt-hash": "sha256:..."
    },
    {
      "stable-id": "lakecat:view:local:default:events_view",
      "view-version": 1,
      "previous-view-version": 1,
      "previous-receipt-hash": "sha256:...",
      "operation": "drop",
      "view-hash": "sha256:...",
      "receipt-hash": "sha256:..."
    },
    {
      "stable-id": "lakecat:view:local:default:events_view",
      "view-version": 2,
      "previous-view-version": 1,
      "previous-receipt-hash": "sha256:...",
      "operation": "upsert",
      "view-hash": "sha256:...",
      "receipt-hash": "sha256:..."
    }
```

```
  ]
}
```

The response is catalog evidence, not Iceberg table metadata. It lets QueryGraph verify the version chain, including tombstones after the current view row is gone, while keeping the richer view history model available for a future Sail-owned implementation.

When the caller needs discovery rather than a known view name, the management surface can return all view receipt chains in a namespace, including chains for views that no longer appear in the active view list:

```
curl -s \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/view-
version-receipt-chains \
  -H 'x-lakecat-agent-did: did:example:resilience-agent'
```

```
{
  "chains": [
    {
      "stable-id": "lakecat:view:local:default:events_view",
      "chain-hash": "sha256:...",
      "chain-verified": true,
      "latest-view-version": 1,
      "latest-operation": "drop",
      "tombstoned": true,
      "receipt-count": 2,
      "receipts": ["..."]
    }
  ]
}
```

That tombstone read is replayable too. LakeCat projects `view.version-receipts-listed` and `view.version-receipt-chains-listed` as lineage evidence, not as graph topology. The graph taxonomy stays in Grust; LakeCat only proves that the governed read saw the tombstone receipt needed to explain why a previously accepted view is now deleted. The namespace response also carries a deterministic `chain-hash` over the chain identity and ordered receipt hashes, and lineage-drain summaries replay that value as `view-version-receipt-chain-hashes` together with a verified-chain count. The QGLake fixture now fails if the namespace-level receipt-chain read, its chain hash, or its verified-chain evidence is absent from lineage-drain replay, so QueryGraph acceptance can depend on compact chain evidence without scraping store internals.

1.23.7 QueryGraph Bootstrap

QueryGraph should import LakeCat facts through a verified handoff, not by scraping service internals. The bootstrap endpoint publishes a bundle with artifact hashes:

The exported graph includes a tenant spine:

```
Catalog HAS_SERVER Server
Server HAS_PROJECT Project
```

```
Project HAS_WAREHOUSE Warehouse
Warehouse HAS_NAMESPACE Namespace
```

When LakeCat has durable management rows, those graph nodes come from the stored `ServerRecord`, `ProjectRecord`, and `WarehouseRecord`. That means a QueryGraph import can see the real server id, project id, warehouse id, display names, and tenant relationships that operators manage through LakeCat's management API. Replay evidence deliberately redacts storage roots and server endpoint URLs into hashes, so QueryGraph can prove which management state was observed without inheriting local paths, bucket roots, query tokens, URI fragments, or userinfo. In the bootstrap graph, `Server` nodes carry `endpointUrlHash` and `Warehouse` nodes carry `storageRootHash`; they do not carry the raw endpoint URL or storage root. The projection code and service route tests both pin those emitted fields as full SHA-256 digest evidence, so the producer and verifier agree on the shape before QueryGraph import. Authorized management responses can still show the configured endpoint URL or storage root to an operator; graph and lineage replay receive hash-only evidence. When those records do not exist yet, LakeCat falls back to the old deterministic default anchors so bootstrap remains compatible with minimal embedded tests and older import flows.

LakeCat also keeps the older `Catalog HAS_NAMESPACE Namespace` edge in the bundle so existing QueryGraph importers can keep working while newer flows read the tenant path. The tenant anchors and warehouse-to-namespace edges are part of the manifest-covered graph hash, so an importer or handoff verifier can reject a bundle whose namespace is silently detached from the warehouse or rebound to a different durable tenant chain. The local QGLake verifier enforces that shape: an accepted bootstrap must prove `Catalog -> Server -> Project -> Warehouse -> Namespace -> Table`, not merely that a table node exists somewhere in the graph. It also rejects bundles whose tenant graph nodes expose raw `endpointUrl` or `storageRoot` properties, even if the graph hash and bundle hash have been recomputed around those raw values. Accepted handoffs must use hash-only `endpointUrlHash` and `storageRootHash` evidence for those roots. Those fields are not merely labels with a `sha256:` prefix: the QGLake verifier requires a full 64-hex SHA-256 digest, so an importer cannot accept a rewritten bundle that replaces raw roots with placeholder hash text after the fact. The saved handoff verifier repeats that check against the archived `lakecat-bootstrap.json` artifact, so a later replay cannot pass with a compact summary whose bundle file has lost the tenant path or drifted from the summary hashes.

```
curl -s \
  -H 'x-lakecat-principal: agent:querygraph-importer' \
  http://127.0.0.1:3000/querygraph/v1/bootstrap \
  -o target/qglake/lakecat-bootstrap.json
```

The bundle contains catalog tables, views, policy bindings, graph artifacts, OpenLineage artifacts, Croissant/CDIF/OSI/ODRL projections, and a manifest that hashes what was emitted. The manifest is the import contract. QueryGraph can refuse a bundle whose graph hash, OpenLineage hash, table artifact hash, view artifact hash, or QueryGraph import-compatibility hash does not match. For view-bearing bundles, the import contract also carries compact receipt evidence for each exported view version:

```
{
  "querygraph-import": {
    "schema-version": "lakecat.querygraph.import-compat.v1",
    "view-count": 1,
    "view-receipt-evidence": [
      {
        "stable-id": "lakecat:view:local:default:events_view",
        "view-version": 1,
        "receipt-hash": "sha256:...",
        "receipt-chain-hash": "sha256:..."
      }
    ],
    "view-receipt-evidence-hash": "sha256:..."
  }
}
```

That gives QueryGraph a manifest-covered way to reject a view bootstrap that lost the accepted catalog receipt or detached the ordered receipt chain before the richer graph import begins.

LakeCat also enforces the same binding when a `querygraph.bootstrap` outbox event is replayed. Table artifact stable IDs must match the `verified-tables` manifest exactly, view artifact stable IDs must match `verified-views`, and view-version receipt stable IDs must match `verified-views`. A saved replay event that keeps valid-looking hashes while swapping in another table artifact or another view's receipt evidence fails before graph or OpenLineage projection.

The QueryGraph side should verify the same bundle before importing it:

```
cd /Users/alexy/src/querygraph/qg-rust

cargo run -- lakecat-verify \
  --bundle /Users/alexy/src/lakecat/target/qglake/lakecat-bootstrap.json

cargo run -- lakecat-import \
  --bundle /Users/alexy/src/lakecat/target/qglake/lakecat-bootstrap.json \
  --output .querygraph/lakecat/import-plan.json
```

The importer checks the outer bundle hash, the manifest hashes, the QueryGraph-import compatibility hash, the graph hash, and view receipt evidence, including the accepted receipt hash and receipt-chain hash for each exported view. The graph envelope must be valid as a graph, not just valid JSON: for example, a table and a view in the same namespace must share one namespace node, not emit duplicate vertex ids. That validation belongs on the QueryGraph/Grust side, while LakeCat is responsible for producing a clean catalog-facing graph projection.

For the full local handoff, LakeCat carries a script that runs both sides without writing generated artifacts into the QueryGraph checkout:

```
scripts/qglake-handoff-local.sh
```

The script starts LakeCat on `127.0.0.1:18181`, uses a Turso-backed local store under `target/qglake-handoff/`, generates the paired QGLake bootstrap bundle and lineage-drain response,

runs `qglake-verify-replay`, then runs QueryGraph's `lakecat-verify` and `lakecat-import` against the same bundle. The resulting import plan is written to `target/qglake-handoff/querygraph-import-plan.json`. The same run also writes `target/qglake-handoff/handoff-summary.json`, a compact machine-readable record under the `lakecat.qglake.handoff-summary.v1` schema. It records the catalog URL, principal, table scope, LakeCat replay status from `lakecat.qglake.replay-verification.v1`, QueryGraph-verified table/view counts, and semantic bundle/graph/OpenLineage/import hashes plus standards accepted only after LakeCat replay, `lakecat-verify`, and `lakecat-import` agree. Before writing that compact summary, the harness also requires the governed scan replay evidence to include planned requested/effective projection, planned requested/effective stats fields, fetched required/effective projection, and fetched filters, with effective projection matching the server-derived read restriction. The compact verifier requires the catalog URL to be an absolute HTTP(S) endpoint and requires warehouse, namespace, and table scope to be present before accepting the summary. It rejects captured QueryGraph verify/import output whose warehouse no longer matches the summary. The lineage-drain replay summaries are bound back to the drain-level `eventTypes` manifest as well. A saved handoff cannot add a compact replay summary for `storage-profile.upserted`, `querygraph.bootstrap`, or any other catalog event type unless the drain itself declared that event type as delivered. LakeCat checks this as a multiset rather than a simple set: repeated event types such as credential vending or scan-task fetching must appear in the same multiplicity in `eventTypes` and in the replay summary array. It also embeds `querygraphVerification.verifiedTables` and `verifiedViews` directly in the compact summary. `verifiedTables` must include the stable LakeCat table id derived from that scope, such as `lakecat:table:local:default:events`; `verifiedViews` must include every accepted stable view id from LakeCat replay, such as `lakecat:view:local:default:active_customers_view`; and both arrays must match the QueryGraph table/view counts. LakeCat rejects duplicate entries in those manifests at outbox admission before graph or OpenLineage projection, and the compact handoff verifier repeats the check for archived summaries. Captured QueryGraph verify/import output must match those compact arrays exactly, which keeps a verified artifact set from being replayed against the wrong catalog tenant, table, or view. The `querygraphImportVerification` object is also a compact proof, not only a boolean: it repeats the QueryGraph import table/view ids, counts, bundle hash, graph hash, OpenLineage hash, QueryGraph import hash, and standards, and LakeCat rejects a summary unless those fields are SHA-256-shaped and match both `querygraphVerification` and the captured `lakecat-import` output. It also records structured request-identity, scan, management, credential, table-commit, and view replay evidence, plus compact `requestIdentityProof`, `queryGraphBootstrapProof`, `governedScanProof`, `tableCommitHistoryProof`, `viewReceiptChainProof`, `managementProof`, `storageProfileUpsertProof`, and `credentialVendingProof` objects that lift the replay principal proof, QueryGraph bootstrap/import proof, governed scan counts, pointer-log read proof, view version and receipt-chain proof, management-list counts, redacted credential-root proof, and credential-vending decision out of the full replay tree. The identity proof shows the principal subject and kind used for the replay, the request-identity source and state, the authorization receipt hash, and sanitized TypeDID envelope/proof hashes when a TypeDID envelope is present. The local QGLake fixture currently records the agent-header source with null TypeDID hash slots; a future

TypeDID-envelope run can fill those slots without changing the handoff schema. The QueryGraph bootstrap proof shows the accepted bundle, graph, OpenLineage, and QueryGraph import hashes, the table and view artifact counts, standards, policy-binding count, agent delegation and summary signature hashes, view-version receipt hashes, and replay/OpenLineage sink hashes. Those core bootstrap hash anchors must also be SHA-256-shaped before the verifier compares them with QueryGraph's verify/import proof. In the compact handoff verifier, the QueryGraph bundle, graph, OpenLineage, import, bootstrap replay, and bootstrap OpenLineage anchors must be full `sha256:`-prefixed 64-hex digests, so saved summaries cannot use prefix-shaped placeholders as QueryGraph acceptance evidence. The scan proof shows LakeCat planned and fetched scan tasks through the governed path, including file, delete-file, and child plan-task counts with replay and OpenLineage hashes. The commit-history proof shows the catalog pointer log was read back with commit count, sequence numbers, commit hashes, positive graph event evidence, and replay/OpenLineage hashes. The view receipt-chain proof shows QueryGraph's accepted view versions together with accepted receipt hashes, accepted `expectedViewVersion` guard evidence when a mutation was guarded, tombstone receipt hashes, namespace chain hashes, verified-chain counts, positive graph event evidence for accepted view replay, and replay/OpenLineage hashes. The credential proof shows the restricted agent was blocked onto Sail-planned reads while the trusted human path used the audited raw-credential exception. The summary also records artifact paths, raw file hashes, captured LakeCat replay output, captured LakeCat handoff-summary verification output, captured QueryGraph verification output, captured QueryGraph import output, captured-output hashes for the LakeCat replay and QueryGraph verify/import JSON files, and service log path. The handoff verifier does not stop at byte hashes: it parses the saved LakeCat replay JSON and QueryGraph verify/import JSON captures and checks their replay schema/status, table and view counts, warehouse, verified table ids, bundle hash, graph hash, OpenLineage hash, QueryGraph import hash, and standards against the compact summary. It compares those standards across sections and independently requires the full QGLake standards set, so a compact handoff cannot omit ODRL or another required contract simply by making every section omit it consistently. It compares the captured LakeCat `replay-evidence.requestIdentity` and `replay-evidence.queryGraphBootstrap` objects with the compact request-identity and bootstrap proofs, including the principal, authorization hash, TypeDID hash slots, delegation and summary signature hashes, artifact counts, standards, replay hashes, and the accepted bundle, graph, OpenLineage, and QueryGraph import hashes. The compact verifier also requires the bootstrap proof to carry the same request-identity source and verification state as `requestIdentityProof`. The authorization receipt hashes are intentionally distinct proof slots: `requestIdentityProof` records the lineage-drain read receipt, while `queryGraphBootstrapProof` records the original bootstrap event receipt. The verifier requires both hashes to be full `sha256:`-prefixed 64-hex digests and bound back to their captured replay sections rather than forcing them to be equal. The same full-digest rule applies to the required agent delegation and agent summary-signature hashes in the bootstrap proof, so a saved handoff cannot replace those proof anchors with short readable placeholders. The compact verifier also validates the TypeDID hash-slot shape directly: envelope and proof slots must be null or full `sha256:`-prefixed 64-hex digests, and a TypeDID proof hash cannot appear without the paired envelope hash. As with authorization receipts, the request and bootstrap TypeDID hash slots are independently shaped replay evidence because they may come

from different requests in the captured workflow. That keeps the compact handoff self-describing without moving TypeDID trust semantics out of TypeSec. Live request parsing now enforces the same boundary earlier: a caller that sends `x-lakecat-typedid-proof` without `x-lakecat-typedid-envelope` receives a hash-only `typedid-proof-hash` rejection, and a caller that sends agent delegation or agent summary proof headers without an agent-shaped identity receives only the matching proof hash. Those failures happen before governance or capability receipt creation, so raw proof material cannot become either policy context or operator-facing diagnostics. TypeDID verifier failures follow the same rule: malformed or rejected envelopes report only the envelope hash and error-detail hash, and a verified-subject mismatch reports only hashes of the verified and supplied principals before governance dispatch. LakeCat applies that redaction at the verifier trait boundary, so a custom TypeDID verifier can choose the error class without leaking raw envelope, DID, gateway, or payload text into the catalog response. The JSON output from `lakecat-cli qglake-verify-handoff` also carries the accepted lineage-drain identity source, identity state, and TypeDID envelope/proof hash slots in `lineageDrainArtifactSemantics`, so QueryGraph can index the verified drain boundary without reparsing the raw drain artifact. Before that boundary is accepted, the verifier reconciles the drain artifact's top-level delivered count, `eventTypes` list, graph event count, and lineage event count against the replay summary array. If a saved `lakecatHandoffVerifyOutput` artifact is present, LakeCat binds those saved drain identity semantics back to the compact `requestIdentityProof`, so a rehash cannot disguise drift in principal, authorization receipt, source/state, or TypeDID hash-slot evidence. The same self-verification pass compares the saved verifier output's delivered count, `eventTypes`, graph event count, and lineage event count with the archived lineage-drain artifact, so a rehashed verifier output cannot rewrite the drain manifest while keeping the artifact hash set intact. It compares captured `replay-evidence.scan` with `governedScanProof`, requiring positive plan task, scan-plan graph event, file task, delete file, and child plan task counts plus the planned and fetched read-restriction objects and the fetch-side required projection/filter evidence. The verifier rejects a summary if the fetched restriction drifts from the planned restriction, so the compact handoff proves the narrowed allowed columns, row predicate, and policy hashes alongside the planned/fetched replay and OpenLineage hashes that prove the Sail-planned read path. The compact Rust verifier requires both planned and fetched replay/OpenLineage arrays to contain full `sha256:`-prefixed 64-hex digests, so automation can reject incomplete or placeholder scan lineage without falling back to the shell harness. Captured scan replay-line recomputation also reuses the governed read-restriction guard, so an archived replay artifact cannot make empty planned or fetched `allowed-columns` look like a readable operator summary. It also compares the captured `replay-evidence.tableCommitHistory` object with `tableCommitHistoryProof`, including the commit count, sequence numbers, commit hashes, replay principal subject/kind, graph event count, replay hashes, and OpenLineage hashes that prove the pointer-log commit history was not rewritten between replay and summary and that the commit-history replay projected catalog graph evidence for the accepted actor. The compact verifier also requires the commit-history principal subject and kind to match the accepted QGLake handoff principal, requires the commit count to match the sequence-number and commit-hash arrays, requires every sequence number to be positive and strictly increasing, requires commit hashes to be duplicate-free, and requires positive graph event evidence plus replay and OpenLineage receipt

hashes. Captured raw lineage-drain regressions cover both missing and drifted commit-history principal subject and principal kind, so actor attribution must survive before the compact handoff proof exists. The service admission layer now rejects `table.commits-listed` source replay whose authorization receipt principal is missing or malformed before acknowledgement, catalog graph projection, or OpenLineage projection, so the raw lineage-drain summary is never built from an actorless pointer-log read. It also binds the replayed warehouse, namespace, and table evidence to the durable outbox table identity before projection, so a source replay cannot project one table's pointer log as another table's history. Captured LakeCat replay-line recomputation enforces the same sequence invariant even when the captured replay JSON and compact summary agree on malformed sequence evidence, so operator-readable `table-commit-history-replay` text cannot launder zero, duplicated, or reordered pointer-log proof. It compares the captured `replay-evidence.views` object with `viewReceiptChainProof`, including accepted view receipts, accepted-view graph event counts, expected-version guard evidence, tombstone receipts, namespace receipt-chain hashes, and their replay/OpenLineage hashes, so durable view history stays tied to the saved LakeCat replay artifact. It also compares the tombstone branch's `expectedViewVersion` with the accepted view version, so a handoff cannot claim a governed deletion unless the saved LakeCat replay proves that deletion used the catalog's optimistic version guard; the standalone `qglake-verify-handoff` command enforces this match even when a summary is checked outside the local shell harness. The same standalone verifier now requires the compact view proof to keep `viewCount` consistent with the accepted view list, carry stable warehouse/namespace/name identity, prove `viewVersion == acceptedViewVersion`, and carry accepted receipt hashes, accepted receipt-chain hashes, positive accepted-view graph event evidence, tombstone receipt hashes, positive verified-chain counts, receipt-chain warehouse/namespace identity, namespace chain hashes, and replay/OpenLineage hashes. It also derives the expected `lakecat:view:<warehouse>:<namespace>:<name>` stable ID from each accepted view's warehouse, namespace, and view name, so a saved handoff cannot keep a verified stable ID while drifting the visible component fields. Tombstone receipt entries use the same component binding before their `expectedViewVersion` guard evidence is accepted, so deletion proof cannot drift from the stable view identity either. The verifier also checks that each namespace receipt-chain summary's `verifiedChainCount` equals the number of structural `chains[]` entries and that every chain entry is covered by `chainHashes`. The compact proof carries `chains[]` entries inside each namespace receipt-chain summary. Each chain entry keeps only catalog-facing evidence: stable view identity, the chain hash, the verified flag, latest view version, latest operation, tombstone state, receipt count, and per-receipt version, operation, view hash, principal subject, principal kind, recorded timestamp, receipt hash, and previous-link fields. The chain warehouse and namespace must match the enclosing namespace receipt-chain summary, and every receipt's stable ID, warehouse, namespace, and view name must match the chain identity. Each structural chain stable ID is also checked against its own warehouse, namespace, and view-name components, so compact evidence cannot splice receipts across views or namespaces while preserving hash-shaped fields. Structural chain bodies cannot repeat a `chainHash`. The enclosing namespace `chainHashes` and `receiptHashes` arrays are duplicate-free summaries of those same structural chains, and `receiptHashes` must match the structural per-receipt hashes exactly, so a compact proof cannot hide extra receipt hashes or omit receipts from the ordered chain bodies. Each

structural `chainHash` is also recomputed from the same content-derived digest LakeCat service uses for view receipt chains: stable view identity, latest version, latest operation, tombstone state, and the ordered receipt hashes. A compact proof therefore cannot pair an accepted receipt-chain hash with a different ordered receipt body. LakeCat enforces the duplicate-free part before compact proof generation as well: outbox replay rejects duplicate `receipt-hashes`, `drop-receipt-hashes`, or `chain-hashes` in view receipt-list and receipt-chain events before graph or OpenLineage projection. Each structural `receiptHash` is recomputed too, using the same content-derived view-version receipt digest LakeCat service emits over stable view identity, version, previous-link fields, operation, view hash, principal, and recorded timestamp. That closes the gap between a valid-looking chain over opaque receipt hashes and a chain whose individual receipt bodies are themselves durable catalog facts. `qglake-verify-handoff` rejects a chain whose first receipt is not version 1 `upsert`, whose previous links do not point to the prior receipt, whose upsert skips a version, whose drop advances the durable version, whose operation is unsupported, or whose latest receipt does not match the chain head. In the compact verifier, accepted-view receipt hashes, accepted receipt-chain hashes, tombstone receipts, namespace receipt/chain hashes, per-receipt hashes, and view replay/OpenLineage hashes must all be full `sha256:`-prefixed 64-hex digests, so a saved handoff cannot use readable placeholder strings as view acceptance evidence. It also requires `queryGraphBootstrapProof.viewVersionReceiptHashes` to match the accepted view receipt hashes exactly, so the compact summary cannot combine bootstrap view receipt evidence from one run with accepted-view proof from another. Accepted receipt-chain hashes and tombstone receipt hashes must be covered by structural `chains[]` evidence for the same stable view, not merely by another chain in the same namespace receipt-chain summary. A tombstoned view therefore carries both the accepted pre-drop chain and a structural drop chain whose final receipt is the tombstone. Tombstoned views must also include tombstone receipt evidence whose `expectedViewVersion` preserves the accepted view version. A consumer can reject a handoff whose view history claim lacks identity, accepted-version, count-aligned hash-chain evidence, duplicate-free exact structural receipt-hash coverage, content-derived chain-hash agreement, same-view accepted-chain coverage, tombstone guard evidence, same-view tombstone receipt coverage, or replay evidence before parsing the full replay tree. It compares captured LakeCat replay `replay-evidence.management` list counts with compact `lakecatReplayVerification.managementProof`, requiring positive server, project, warehouse, and storage-profile counts, positive graph event counts for server, project, warehouse, policy-binding, and storage-profile list replay, and a policy-binding count that matches the QueryGraph bootstrap proof. Those management-list counts are receipt-backed: the compact proof and captured replay must also agree on replay and OpenLineage hash arrays for `server.listed`, `project.listed`, `warehouse.listed`, `policy-binding.listed`, and `storage-profile.listed`. It also compares the captured LakeCat replay `replay-evidence.management.storageProfileUpsert` object with the compact `lakecatReplayVerification.storageProfileUpsertProof`, including the profile id, provider, issuance mode, location-prefix hash, secret-reference presence/provider/hash, replay hashes, and OpenLineage hashes. The compact verifier also requires those replay and OpenLineage arrays to contain full `sha256:`-prefixed 64-hex digests. It requires that location-prefix value to be a full `sha256:`-prefixed 64-hex digest and requires a redacted secret-reference provider plus full-digest

secretRefHash whenever the proof says a secret reference is present. If the proof says no secret reference is present, the provider and hash may be omitted or null, but any other provider/hash value is rejected. Source replay enforces the same full-digest secret-reference rule before compact proof generation, so the saved summary cannot launder short placeholder credential-root hashes through the lineage-drain artifact. The positive QGLake acceptance fixture covers the production-shaped case too: when the storage profile uses a secret reference, the compact handoff accepts the proof only when the storage-profile branch and both credential branches agree on the redacted provider and full secretRefHash, and the operator replay line carries secret_ref_hash=sha256:... rather than a raw secret URI. Those operator-facing management and credential replay lines also fail closed when secret-backed evidence has only a prefix-shaped or placeholder hash, so the human-readable proof cannot be weaker than the structured verifier. It also compares the captured replay-evidence.credentials restricted-agent and trusted-human branches with the compact credentialVendingProof, so a saved handoff cannot claim that agents were blocked onto Sail-planned reads or that humans used an audited exception unless the captured LakeCat replay proves the same decision. That equality includes credentialPrefixHashes, which closes the gap where a captured replay artifact could report a different redacted returned-credential set while the compact summary still looked valid. Both credential branches must carry replay and OpenLineage arrays whose entries are full sha256:-prefixed 64-hex digests, so the compact proof cannot replace credential receipt evidence with prefix-shaped placeholders. They also carry credentialPrefixHashes: the restricted-agent branch must prove the array is empty when zero credentials were returned, while the trusted-human branch must prove the array length matches credentialCount, every entry is a full SHA-256 digest, and no prefix hash is repeated. The verifier also binds the operator-facing replay text back to the same proof fields. The captured top-level scan-replay line is recomputed from governedScanProof, including plan/fetch task counts, the policy-derived purpose, and the TTL cap. The captured top-level credential-replay line is recomputed from credentialVendingProof, including the restricted-agent block, trusted-human exception, TTL caps, and redacted storage-profile anchors. The captured management-replay line is also recomputed from managementProof and storageProfileUpsertProof, while table-commit-history-replay is recomputed from tableCommitHistoryProof. A saved artifact therefore cannot keep valid structured proof while presenting a different terminal transcript or principal-attribution story to an operator. Each credential branch carries the same redacted storage-scope anchor as the storage-profile upsert proof: locationPrefixHash binds the credential-vend attempt to the configured storage root without replaying the raw prefix. That anchor must be a full sha256:-prefixed 64-hex digest in both the storage-profile proof and each credential branch. LakeCat checks the storage-scope hash at lineage-drain replay time before the compact handoff summary is accepted, and the operator-readable credential replay line prints the same hash so captured terminal output cannot look complete while omitting the credential-root boundary. Source replay validates secret-reference shape on the credential branches themselves: if a credential-root proof says a secret reference is present, it must carry a non-empty provider and full sha256:-prefixed 64-hex secretRefHash; if it says no secret reference is present, provider and hash evidence must be absent. LakeCat now applies the same discipline before the outbox event is delivered: credentials.vend-attempted must carry a credential-count that matches its credential-response evidence, full SHA-256

prefix and issuer-config hashes for each returned credential, a full storage-profile `location-prefix-hash`, and non-contradictory secret-reference state. The top-level `storage-profile-id` must match the nested `storage-profile.profile-id`, even when no raw credentials were returned, and the nested `storage-profile.warehouse` must match the event table warehouse. The replay payload's `table` hint must also match the durable outbox table identity before acknowledgement, graph projection, or OpenLineage projection, so a credential decision for one table cannot be replayed as another table's credential-root evidence. If the top-level `secret-ref-present` field is present, it must match `storage-profile.secret-ref-present`; older replay fixtures may omit the duplicate field, but contradictory evidence is rejected. Each returned credential entry must also agree with the catalog-derived storage-profile id, catalog profile id, storage provider, credential mode, authorization principal, receipt principal, governed-read marker, and any policy-derived TTL cap. Returned credential entries must be duplicate-free by `prefix-hash`, so a replay event cannot count the same redacted credential twice. LakeCat carries those redacted prefix hashes into the raw lineage-drain summary as `credentialPrefixHashes`, and the QGLake verifier rejects the drain before compact proof generation if the prefix hashes are missing, count-drifted, short, or duplicated. A malformed credential replay event therefore remains pending instead of becoming graph or OpenLineage evidence. Credential replay also rejects a governed `read-restriction` that is missing from, or different from, the authorization receipt context, so credential TTL and blocked-agent evidence cannot drift away from the receipt that authorized the decision. Source replay and compact handoff verification both reserve `rawCredentialExceptionReason` for the audited trusted-human path; a restricted agent proof must be blocked with `blockReason` and cannot carry a raw exception reason. The local handoff harness now preserves the lineage-drain artifact before replay verification, so if any of these proof checks fail the operator still has the exact failed drain JSON for diagnosis. The compact handoff verifier also validates that credential proof directly: the restricted branch must name the accepted agent principal, carry the Sail-planned-read block reason, prove zero credentials, carry the policy-derived `maxCredentialTtlSeconds` cap, explicitly set `rawCredentialExceptionAllowed` to false, reject any non-null `rawCredentialExceptionReason`, and include replay/OpenLineage hashes; the trusted-human branch must name a human principal, prove a positive credential count, carry the same policy-derived TTL cap, carry the exact audited raw-credential exception reason, prove `blockReason` is null, and include replay/OpenLineage hashes. Both branches must also carry count-aligned, duplicate-free `credentialPrefixHashes`, with an empty array for the blocked branch and full SHA-256 returned-prefix hashes for any issued credential. The compact verifier has direct negative coverage for the credential-branch secret-reference rules too: blank providers are rejected when a secret ref is present, and non-null provider/hash evidence is rejected when no secret ref is present. That way a handoff cannot hide malformed provider or hash evidence behind an otherwise matching storage-profile upsert proof. The local handoff harness now preserves those replay fields while building the compact summary, rather than relying on the nested raw replay blob alone: scan graph events and fetch-side projection/filter requirements, management graph events, storage-profile graph events, credential block/exception fields, and table commit-history graph events all move into the verifier-facing proof sections before `qglake-verify-handoff` runs. That makes the handoff repeatable from the LakeCat repo while keeping QueryGraph responsible for graph validation and import semantics. The handoff script refuses

to write the summary unless LakeCat replay JSON contains request-identity evidence for the expected agent principal, an explicit identity source/state, an authorization receipt hash, and explicit TypeDID envelope/proof hash slots that are either null or full `sha256:`-prefixed 64-hex digests. A proof hash is valid only when the matching envelope hash is present. It also refuses to write the summary unless LakeCat replay JSON contains redacted `storageProfileUpsert` evidence with replay and OpenLineage hashes, and the accepted summary repeats that evidence as `lakecatReplayVerification.storageProfileUpsertProof`. QueryGraph gets proof that the credential root was configured, including the provider, issuance mode, the configured location-prefix hash, and a full `sha256:`-prefixed 64-hex digest of the secret reference, without receiving the underlying secret-store URI or full storage prefix in the compact proof. The operator-readable management replay line now prints the same storage-scope hash and redacted secret-reference state, so a captured transcript cannot describe the credential root only by provider while omitting its redacted storage scope or secret-reference boundary. The saved handoff verifier recomputes that management line from compact proof fields before accepting captured output. The script also refuses to write a summary unless LakeCat replay proves both sides of credential vending: untrusted agents get no raw credentials, trusted humans receive only the audited standard exception, and both branches preserve the `max-credential-ttl-seconds` restriction as `maxCredentialTtlSeconds` in compact evidence. For reads, the summary similarly refuses to omit proof that scan planning and scan-task fetch both replayed with full digest-shaped sink receipt hashes. The compact scan proof must preserve the server-derived read restriction as a full restriction, not only as columns and filters: allowed columns, row predicate, purpose, full `sha256:`-prefixed policy-hash evidence, and `max-credential-ttl-seconds` must be present, and the planned and fetched restrictions must agree. Short readable policy anchors such as `sha256:policy-name` are rejected before QueryGraph receives the compact handoff proof. The fetched required filters must also be exactly the mandatory row predicate evidence, not a prefix with extra unverified filters appended. For catalog state, it refuses to omit proof that the table commit-history read replayed with sequence-number and commit-hash evidence. For views, it refuses to omit proof that accepted view versions line up with their receipt hashes and that the namespace-level tombstone and receipt-chain reads replayed with chain hashes and verified-chain counts. The service-side lineage-drain summary preserves the full receipt hash set from receipt-list reads and nested namespace receipt-chain payloads, so the QGLake verifier can prove that both upsert and tombstone receipts are covered by the replayed namespace chain.

This gives the semantic layer a responsible starting point. LakeCat says:

```
Here are the governed catalog objects.
Here are the policies that shaped planning.
Here are stable dataset and field anchors.
Here is the graph envelope.
Here is the OpenLineage replay evidence.
Here are the hashes that bind the handoff.
```

QueryGraph then owns the richer semantic work: metrics, dimensions, joins, business names, multi-dataset reasoning, and agent-facing synthesis.

1.23.8 Draining The Outbox

LakeCat records side effects as durable outbox events. Draining the outbox is what turns committed catalog facts into graph and lineage receipts:

```
curl -s -X POST \
  -H 'x-lakecat-principal: agent:lineage-drainer' \
  http://127.0.0.1:3000/management/v1/lineage/drain \
  -o target/qglake/lineage-drain.json
```

A useful drain response includes delivered event types, graph projection counts, lineage projection counts, receipt hashes, and the authorization proof for the drain request itself. That last part is easy to overlook. Reading the replay stream is also privileged, so LakeCat records that the drainer was allowed to read lineage evidence. Standard catalog reads replay too: `catalog.config-read` records a warehouse-scoped graph/OpenLineage fact for the Iceberg REST configuration entrypoint; `namespace.listed` records the namespace listing at the warehouse; and `namespace.loaded` records the specific namespace resolved through the standard catalog route. For view events, the response includes the warehouse, namespace, view name, and QueryGraph-compatible stable id, so downstream acceptance can check replay identity without parsing the full audit payload. Table restores replay as table graph evidence plus a restore OpenLineage receipt, so a soft-deleted table returning to service is visible to QueryGraph without forcing LakeCat to invent restore-specific graph taxonomy. Management list reads for policy bindings, projects, servers, storage profiles, and warehouses replay as OpenLineage receipts too. They intentionally do not create list-specific graph nodes in LakeCat; Grust owns the reusable hierarchy and traversal model. The durable audit/outbox payload carries only the redacted stable ID arrays beside the counts: policy ids, project ids, server ids, storage-profile ids, and warehouse names. Before projection, LakeCat rejects a management-list event when the required ID array is missing, malformed, contains an invalid identifier, repeats an identifier, or no longer matches the recorded count. LakeCat also requires the authorization receipt to carry a valid principal before acknowledging the event, so QueryGraph never has to accept actorless management inventory replay. The drain response lifts their counts, ID arrays, and management scope into compact fields, so QueryGraph can verify the control-plane read evidence without opening the raw lineage payload. It also carries replay and OpenLineage hash arrays for those management-list reads, so a compact handoff cannot prove only that the right number of management records existed while losing the identities or receipt evidence for the reads. The lineage-drain verifier rejects those source replay events when the ID arrays are missing, empty, duplicate-inflated, count-drifted, or when the receipt arrays are empty or not SHA-256-shaped, so the compact `managementProof` starts from verified replay evidence rather than untrusted text. The compact handoff verifier repeats that check with the stricter full `sha256:`-prefixed 64-hex digest shape for every management replay and OpenLineage array, and it verifies that `serverIds`, `projectIds`, `warehouseNames`, `policyIds`, and `storageProfileIds` match their recorded counts and are duplicate-free. Saved summaries therefore cannot preserve only prefix-shaped placeholders for control-plane read receipts, inflate a count with repeated valid identities, or normalize malformed management identities later. Captured replay agreement checks the same ID arrays against the saved compact `managementProof`, so a handoff cannot keep valid artifact hashes while swapping the server, project, warehouse,

policy, or storage-profile identities between source replay and the summary. When an operator preserves the verifier output as `lakecat-handoff-verify.json`, LakeCat re-checks the saved `capturedOutputSemantics.lakecatReplay` proof against the compact summary for every replay section, including management IDs, governed scans, commit history, view receipt chains, storage-profile evidence, and credential-vending proof. That makes the verifier output a replayable audit artifact instead of a second place where compact proof drift can hide. It also rejects management-list source replay without catalog graph projection evidence, keeping the durable server/project/warehouse, policy, and storage-profile facts visible to QueryGraph through Grust-facing graph events. Compact `managementProof` carries those graph event counts too, and captured replay agreement checks them, so the graph evidence cannot disappear between source replay and handoff verification. The QGLake acceptance workflow now establishes its server/project/warehouse tenant spine, performs governed server, project, warehouse, policy-list, storage-profile-list, scan-planning, scan-task-fetch, and table commit-history reads before bootstrap, and rejects a drain that does not replay matching `server.listed`, `project.listed`, `warehouse.listed`, `policy-binding.listed`, `storage-profile.listed`, `table.scan-planned`, `table.scan-tasks-fetched`, and `table.commits-listed` evidence. Request-identity and bootstrap replay are checked before any compact handoff proof is built: the drain authorization, bootstrap authorization, QueryGraph bundle/import hashes, agent delegation hash, agent summary signature hash, and TypeDID envelope/proof hashes must be full `sha256:`-prefixed 64-hex digests, and a TypeDID proof hash cannot appear without its paired envelope hash. For scan replay, the typed drain summary carries scan-plan task counts, scan-plan graph event evidence, fetched file-scan, delete-file, and child-plan task counts, along with planned and fetched OpenLineage receipt hashes. Source replay validation now also requires planned/fetched replay and OpenLineage receipt arrays to be full SHA-256 digests, and the compact handoff verifier repeats that full-digest check for the saved `governedScanProof` arrays. The scan read restriction itself is part of that proof: both source replay and compact `plannedReadRestriction`/`fetchedReadRestriction` evidence require `policy-hashes` to be non-empty full `sha256:`-prefixed 64-hex digests, so a self-consistent handoff cannot smuggle placeholder policy names or empty policy anchors through a field that later readers treat as integrity evidence. The outbox drain checks the same digest shape and non-empty requirement before acknowledging any pending event that carries `read-restriction.policy-hashes`, including the copy embedded in the authorization receipt context, so malformed source evidence is stopped before it becomes delivered replay material. Scan replay also requires the top-level read restriction to match the authorization receipt context exactly before delivery, so policy narrowing cannot be asserted in one replay field and absent from the durable receipt. Scan replay now gets the same drain-side admission check before Grust or OpenLineage projection: planned-scan events must carry matching table identity, unsigned task counts, requested/effective projection arrays, and requested/effective stats-field arrays; fetched-task events must carry matching table identity, fetched file/delete/child-plan counts, required filters, and required/effective projection arrays. Those scan proof arrays must be non-empty, non-blank, and duplicate-free; present-but-empty projection or stats evidence is malformed, not an implicit unrestricted read. Fetched-task `required-filters` must also exactly preserve the governed row predicate at service admission, so an event with empty or drifted fetched filter proof is rejected before graph or OpenLineage projection. When a governed read restriction is present, the effective projection and effective

stats fields must remain inside the allowed columns, empty allowed-column arrays fail closed for both planned and fetched replay, and explicit effective projection or stats evidence cannot widen beyond the caller-requested or server-required evidence it claims to preserve. QueryGraph bootstrap replay is also checked at the drain boundary before it becomes accepted handoff material: the event must carry a valid warehouse, table/view counts matching the verified ids and artifact arrays, full SHA-256 bundle/graph/OpenLineage/import hashes, full table/view artifact hashes, view receipt and receipt-chain hashes for accepted views, the expected standards list, and full optional TypeDID or agent proof hashes when those slots are present. View receipt replay follows the same fail-closed rule at the drain boundary. A `view.version-receipts-listed` event is not acknowledged unless its `receipt-count` matches full SHA-256 receipt hashes and every drop receipt hash is included in the listed receipts. A verified `view.version-receipt-chains-listed` event is not acknowledged unless its verified-chain count matches the chains marked verified, each verified chain and receipt carries full SHA-256 digest evidence, the first receipt is a version 1 upsert without previous links, and every later upsert or drop links to the previous receipt with the expected view-version transition. That keeps malformed view-history evidence out of both graph projection and OpenLineage replay before QueryGraph ever sees a compact handoff. The verifier also requires table-commit replay to be internally consistent before delivery: `table.commit` must carry a commit object, unsigned sequence number, stable table identity, matching nested commit-table identity, a valid commit principal and a valid authorization receipt principal with matching values, and full SHA-256 commit hash evidence before graph or OpenLineage projection can start. Commit-history replay has the same shape: `table.commits-listed` event must carry a `commit-count` that matches both full SHA-256 commit hashes and unsigned sequence numbers; compact QGLake proof also binds that pointer-log replay to the accepted principal subject/kind. The raw QGLake lineage-drain verifier checks the same accepted-principal and agent kind before compact handoff proof is generated, so malformed or actor-drifted pointer-log summaries cannot become delivered replay evidence. Credential-vend replay gets the same treatment: `credentials.vend-attempted` must carry a matching credential count, full duplicate-free credential-response prefix hashes, a full redacted storage-profile location hash, internally consistent secret-reference presence/provider/hash fields, a top-level storage-profile id that agrees with nested storage-profile evidence, a nested storage-profile warehouse that agrees with the event table warehouse, any top-level secret-reference presence value that agrees with nested storage-profile evidence, and credential-response metadata that agrees with the selected storage profile and authorization receipt before delivery. Storage-profile upsert replay must likewise reject raw secret references and contradictory secret-reference-state evidence before delivery. Policy-binding upsert replay must carry valid catalog scope evidence before delivery, including policy id, warehouse, optional namespace/table scope, enforcement state, and captured ODRL material. Namespace lifecycle replay must carry a valid warehouse and namespace path or component array before create/load/drop events can be delivered. Catalog config and namespace-list read replay must likewise carry a valid warehouse, and namespace listing must preserve both an unsigned namespace count and count-aligned `namespace-paths` evidence. Those paths are parsed as namespace identities and must be non-empty and duplicate-free before standard catalog reads become delivered graph/OpenLineage evidence. Catalog config, namespace list, namespace lifecycle, view list, and view lifecycle replay must also carry valid authorization receipt principals, so saved replay cannot

turn standard Iceberg control-plane activity into actorless QueryGraph facts. Management-list read replay applies the same rule to operational discovery: policy, project, server, storage-profile, and warehouse list events must carry unsigned counts, valid warehouse scope when warehouse-scoped, and valid optional project scope before delivery. View list and lifecycle replay must carry valid warehouse, namespace, view name, count, and receipt principal evidence before those view events become graph/OpenLineage material for QueryGraph. Table lifecycle replay applies the same identity discipline before delivery: `table.created`, `table.loaded`, `table.deleted`, and `table.restored` must carry a decodable table identity, optional payload scope hints must match it, and soft-delete evidence must point at the same table with an unsigned version. Project, server, and warehouse upsert replay must carry valid tenant-root evidence too: project ids, server scopes, endpoint URLs, storage roots, identifiers, properties, and pre-redacted hash anchors are checked before delivery. Policy-binding, project, server, storage-profile, and warehouse upsert replay must also carry a valid authorization receipt principal before delivery, so compact QueryGraph proof can attribute management mutations to the actor accepted by LakeCat. It also requires planned and fetched read restrictions to match before compact proof generation, requires both requested/effective projection and requested/effective stats-field evidence, requires effective projection to be a narrowed subset of the requested projection and to match the planned allowed columns, and requires effective stats fields to be both inside the planned allowed columns and a narrowed subset of the requested stats fields in source replay and compact handoff proof. Empty allowed-column evidence is rejected in planned and fetched replay instead of being interpreted as unrestricted replay. It also requires the fetched projection and filter requirements to exactly preserve the fetched allowed columns and row predicate. A fetched response that omits required-filter proof is rejected just like one that widens or changes that proof, and the compact handoff summary applies the same missing-proof check before accepting governed scan evidence. Credential replay applies the same policy-proof discipline to the two credential branches: the restricted-agent denial and trusted-human audited raw-credential exception must both carry a complete read restriction, and those restrictions must match before credential proof can feed the compact handoff summary. For commit-history replay, the typed drain summary carries the commit count, committed sequence numbers, commit hashes, replay hashes, and OpenLineage hashes. The handoff verifier rejects compact scan proofs without those OpenLineage hashes and compact commit-history proofs whose counts, sequences, or hash arrays do not align. It also requires the compact commit, replay, and OpenLineage arrays to contain full `sha256:`-prefixed 64-hex digests, not prefix-shaped placeholders. Source replay validation applies the same pointer-history discipline before compact proof generation: the table commit count must match the sequence-number and commit-hash arrays, commit sequences must be positive and strictly increasing, and commit hashes must be SHA-256-shaped before pointer-history evidence can enter the compact handoff proof. Service route coverage pins the producer side too: request hashes, response hashes, idempotency-key hashes, and commit hashes are full SHA-256 digests across the route response, pointer-log outbox payload, lineage-drain summary, and graph projection. The QGLake fixture verifier also checks the management commit-history response itself, so short readable placeholders are rejected before the later lineage-drain and compact handoff checks run. QueryGraph can therefore verify the governed Sail-planned read and pointer-history inspection without parsing the full lineage payload. The core QueryGraph bundle, graph, OpenLineage, and import

anchors must be SHA-256-shaped in compact verify/import/bootstrap proof before a matching summary can pass; matching strings are not enough. The bootstrap, scan, credential, view, receipt-chain, and commit-history receipt arrays must also be SHA-256-shaped before compact proof generation can consume them. The same shape check applies to accepted-view receipt evidence: bootstrap view-version receipt hashes, tombstone receipt hashes, and namespace receipt-chain hashes must be full SHA-256 digest evidence before accepted-view proof feeds the compact handoff summary. Dropped and active accepted-view source replay also compares the bootstrap view-version receipt hashes with the accepted QueryGraph verification set, so a valid-looking receipt array cannot be spliced from another bootstrap proof. The compact handoff verifier repeats the same binding against `viewReceiptChainProof.views[].acceptedReceiptHash` and `viewReceiptChainProof.views[].acceptedReceiptChainHash`, including tombstoned accepted views, catching drift even when an operator validates only the saved summary. Dropped accepted-view source replay also binds the namespace receipt-chain read back to the accepted view warehouse/namespace and rejects verified-chain count or receipt-hash coverage drift before compact handoff proof is generated. The lineage-drain summary now carries the nested chain receipts as full receipt hash coverage before that check runs. Generated replay evidence also preserves each accepted view's `acceptedReceiptChainHash` inside the namespace `receiptChains[].chainHashes` set, even when the namespace read has its own chain hash, so the compact summary can prove the accepted chain is covered by the namespace proof it verifies. The same replay now emits catalog-facing `Commit` graph events for the listed sequences, leaving traversal and query semantics to Grust.

For handoff testing, LakeCat can verify a saved bootstrap bundle and a saved drain response together:

```
cargo run -p lakecat-cli -- qglake-verify-replay \
  --bundle target/qglake/lakecat-bootstrap.json \
  --drain target/qglake/lineage-drain.json \
  --principal did:example:agent
```

That offline check replays the same boundary assertions used by the live QGLake fixture: the bundle manifest must verify, the QueryGraph import compatibility contract must match, and the lineage drain must carry matching bootstrap hashes, credential-denial receipts, management-list evidence, scan/fetch task evidence, and table commit-history receipt evidence, plus view receipt evidence when views are present. On success, the command prints the accepted bundle and QueryGraph import hashes, table/view counts, and compact control-plane lines such as:

```
scan replay plan_tasks=1 plan_graph_events=1 planned_ttl=300
planned_purpose=qglake-agent-demo file_tasks=1 delete_files=1 child_plan_tasks=1
fetched_ttl=300 fetched_purpose=qglake-agent-demo
management replay servers=1 projects=1 warehouses=1 policies=1 storage_profiles=1
storage_profile_upserts=1 credential_root=events-local:file:local-file-no-
secret:location_prefix_hash=sha256:2222222222222222222222222222222222222222222222222222222222222222
credential replay restricted=blocked:sail-planned-read-required
restricted_count=0 restricted_ttl=300 restricted_profile=events-local:file:local-
file-no-
secret:location_prefix_hash=sha256:2222222222222222222222222222222222222222222222222222222222222222
```

```
human=allowed:trusted-human-audited-raw human_count=1 human_ttl=300
human_profile=events-local:file:local-file-no-
secret:location_prefix_hash=sha256:2222222222222222222222222222222222222222222222222222222222222222
table commit history commits=1 sequences=1 hashes=sha256:... graph_events=1
```

Those lines are intentionally small enough for QueryGraph handoff scripts and operator logs, but they still come from the same typed lineage-drain summaries that the verifier requires before accepting replay. The saved handoff verifier recomputes each line from compact proof fields, including the management credential-root anchor and the table commit-history sequence/hash summary, before accepting captured LakeCat replay output. LakeCat service replay now requires table commit-history sequence numbers to be positive and strictly increasing, and commit hashes to be duplicate-free, before graph or OpenLineage projection. The compact pointer-log summary therefore cannot inherit duplicated or reordered catalog evidence.

The scan line keeps the planned and fetched credential TTL caps visible beside the task counts, while JSON mode carries the full read-restriction evidence tree. Scan planning records both requested and effective projection evidence; scan-task fetch records the server-derived required projection and mirrors it as `effective-projection`, so replay can compare both stages with the same policy-narrowed vocabulary. QGLake acceptance now rejects handoffs where the fetched effective projection is missing or drifts away from the fetched read restriction, which means a compact replay summary cannot quietly widen what the server actually planned. The live handoff harness performs that projection check before writing `handoff-summary.json`, so the artifact is born with the same proof shape the verifier later enforces.

After the full local handoff writes `handoff-summary.json`, LakeCat can also verify the compact summary itself:

```
cargo run -p lakecat-cli -- qglake-verify-handoff \
  --summary target/qglake-handoff/handoff-summary.json \
  --json
```

That command validates the `lakecat.qglake.handoff-summary.v1` schema, QueryGraph verify/import agreement, LakeCat replay agreement, and the compact proof objects for request identity, QueryGraph bootstrap, governed scan, pointer history, view receipt chains, storage-profile upsert, and credential vending. It also recomputes the raw file hashes for the bundle, lineage-drain response, and QueryGraph import plan named in the summary, rejecting stale or tampered artifact files before automation consumes them. Those declared artifact hashes must be full `sha256:`-prefixed 64-hex digests, so a handoff cannot present readable placeholder hashes as structurally valid integrity anchors before the byte comparison runs. It parses the saved bootstrap bundle and reruns the tenant graph and semantic hash verifier. It also parses the saved QueryGraph import plan and requires its embedded verification, table/view stable ids, semantic hashes, standards, and graph node/edge evidence to match the compact QueryGraph import proof. The verifier requires the compact `verifiedTables` and `verifiedViews` manifests to be duplicate-free as well as count-aligned, matching service-side outbox admission, so a saved handoff cannot inflate the number of accepted tables or views by repeating an already accepted stable id. Raw lineage-drain replay summaries and compact handoff proof sections both keep replay, OpenLineage, credential

prefix, view receipt, and view receipt-chain hash arrays duplicate-free, not only `sha256:` shaped. That covers the bootstrap, governed scan, management, table commit-history, view tombstone/receipt-chain, storage-profile upsert, and credential-vending proof sections, so a source replay or archived handoff cannot make an evidence set look larger by repeating an already accepted digest. The verifier also compares those QueryGraph import-plan graph node and edge counts with the verified bootstrap bundle graph counts, so an import plan cannot keep the semantic hashes and table/view ids while silently dropping graph material. It also parses the saved lineage-drain response, reruns the typed QGLake replay verifier, and regenerates the LakeCat replay evidence that proves request identity, QueryGraph bootstrap replay, governed scan replay, pointer history, view receipt chains, storage-profile replay, and credential-vending decisions. It then compares that regenerated replay evidence to the compact `lakecatReplayVerification` proof. The governed scan proof includes both the requested and effective stats-field arrays from the scan-planned replay, and the verifier rejects handoffs where the effective fields no longer match the allowed columns or no longer prove policy narrowing. Credential-vending proof is not just a count: each restricted-agent and trusted-human branch carries the redacted `storageProfile` graph anchor and `maxCredentialTtlSeconds`, including profile id, provider, issuance mode, secret-reference presence, and the graph event count emitted by replay. The storage-profile upsert proof also carries its own positive `graphEvents` count, and captured LakeCat replay must match it. The verifier also rejects a handoff when the credential branches do not bind back to the same storage-profile upsert proof: profile id, provider, issuance mode, location-prefix hash, and secret-reference state must all match the replayed management event. A saved handoff is rejected if the archived lineage drain proves lineage receipt hashes but omits that credential TTL cap or credential-root graph projection. It also recomputes the captured LakeCat replay and QueryGraph verify/import output hashes, so terminal captures cannot drift from the compact summary. It compares the legacy string path aliases for the LakeCat replay, QueryGraph verify, and QueryGraph import captures with the hashed `capturedOutputs` paths they duplicate. It also hashes the service log through a full-digest `serviceLogHash`, so archived operational logs cannot drift behind a stable path or a short placeholder hash. The final local summary also binds the first LakeCat handoff-verifier capture with a full-digest `lakecatHandoffVerifyOutputHash`. Because that output can only exist after a successful verifier run, the harness performs a second sidecar self-check: first it writes `target/qglake-handoff/lakecat-handoff-verify.json`, then it records the file's hash in the summary, then it verifies the summary again without overwriting the declared artifact. The verifier checks that saved JSON is a `lakecat.qglake.handoff-verification.v1` success for the same principal, catalog URL, warehouse, namespace, and table, and that its table/view counts, stable ids, standards, request-identity proof, and QueryGraph bootstrap proof still match the compact handoff summary. It also checks the saved self-verifier output's bundle, lineage-drain, QueryGraph import-plan, captured-output, and service-log hashes against the summary's artifact manifest. It also checks the saved self-verifier output's own semantic sections: captured replay semantics must match the compact LakeCat and QueryGraph proof, bundle artifact semantics must match QueryGraph verification, import-plan semantics must match QueryGraph import verification, lineage-drain semantics must match the accepted replay proof, and saved import-plan graph counts must still match the saved bundle graph counts. Then it parses the archived lineage-drain artifact and requires the saved lineage-drain semantics' delivered count, event type

list, graph event count, and lineage event count to match before accepting the verifier-output hash. The archived drain itself must also reconcile those same top-level counts with its replay summary array, including repeated event-type multiplicity. Then it parses those captured JSON files and checks that the replay schema/status, table/view counts, semantic hashes, standards, request-identity proof, QueryGraph bootstrap proof, governed scan proof, storage-profile upsert proof, and credential-vending proof inside the captures still match the summary. It also rejects malformed TypeDID hash slots in the request-identity and QueryGraph bootstrap proofs before a consumer has to interpret those slots. The local handoff harness runs it automatically and writes the captured verifier output to `target/qglake-handoff/lakecat-handoff-verify.json`.

The end-to-end result is a chain:

```
catalog write
  -> audit event
  -> outbox event
  -> graph projection
  -> OpenLineage projection
  -> QueryGraph import evidence
```

If graph or lineage sinks are down, catalog state should not be lost or rolled back accidentally. The outbox lets LakeCat retry projection from committed state. A drain acknowledges delivery only after every projection in the batch succeeds. If OpenLineage fails after a graph event has already been emitted, the drain fails and the catalog event remains pending, so recovery starts from the committed outbox rather than from guesswork. If the graph sink fails first, LakeCat fails the drain before emitting lineage and still leaves the outbox event pending, so graph and lineage consumers recover from the same committed catalog fact instead of diverging.

Replay order is part of that contract. LakeCat selects undelivered outbox events by `created_at,event_id` in both embedded memory tests and the durable Turso store, so a QGLake replay does not depend on writer interleaving or database row-return quirks. Delivery acknowledgement is duplicate-safe as well: if a drainer accidentally reports the same event id twice, LakeCat marks the event once and the receipt count remains tied to committed catalog facts. Pending batch validation happens before projection: if a store returns duplicate event IDs in the same drain batch, LakeCat fails the drain with only the duplicate event-id hash and does not emit graph, OpenLineage, or acknowledgement side effects for that batch. The same redaction rule applies to malformed pending records. If a custom or corrupted store hands the drain an event whose payload cannot be projected, LakeCat reports only the outbox event-id hash and stops before graph emission, lineage emission, or delivery acknowledgement. Malformed table and principal identity JSON decode failures, as well as unsupported event types, follow that same pattern: they carry event-hash evidence for correlation without echoing the raw event identifier into diagnostics.

1.23.9 An Agentic QGLake Flow

The agentic path is the reason LakeCat has to be more than a passive catalog. Imagine a resilience supervisor agent investigating incidents:

1. The supervisor delegates table triage to a specialist agent.
2. The specialist asks LakeCat to plan a scan over `local.default.events`.
3. LakeCat resolves the agent identity and TypeDID context.
4. TypeSec authorizes the table scan and returns a restricted capability.
5. LakeCat narrows the projection and appends the required row predicate.
6. Sail plans against the current Iceberg metadata and delete manifests.
7. LakeCat returns governed plan and fetch-task responses.
8. The specialist summarizes only the allowed result shape.
9. LakeCat records scan and credential decisions into audit/outbox.
10. QueryGraph imports graph, policy, lineage, and bootstrap evidence.

The key point is the absence of raw storage reach. The specialist agent does not need broad cloud credentials to do its job. It needs a governed plan, a bounded task set, and a receipt trail.

The local fixture compresses this story into a short artifact-producing sequence:

```
cargo run -p lakecat-cli --features qglake-fixture -- qglake-fixture \
  --output target/qglake/lakecat-bootstrap.json \
  --drain-output target/qglake/lineage-drain.json \
  --principal did:example:agent
cargo run -p lakecat-cli -- qglake-verify-replay \
  --bundle target/qglake/lakecat-bootstrap.json \
  --drain target/qglake/lineage-drain.json \
  --principal did:example:agent
```

The fixture generator opts into `lakecat-cli`'s `qglake-fixture` feature because it writes local Iceberg metadata and manifest files through Sail. Verification commands stay in the default CLI surface, which keeps ordinary catalog inspection, replay, and handoff checks available without pulling in the local fixture writer.

The one-command handoff wraps the same evidence in a live local service run and then asks QueryGraph to verify and import it:

```
scripts/qglake-handoff-local.sh
cargo run -p lakecat-cli -- qglake-verify-handoff \
  --summary target/qglake-handoff/handoff-summary.json \
  --json
cat target/qglake-handoff/handoff-summary.json
```

That fixture creates the sample table shape, installs a restricted policy, verifies governed scan planning, verifies fetch-scan-task reapplication, checks requested/effective stats-field narrowing in replay and handoff proof, exercises delete manifest handling, probes credential-vend behavior for agents and trusted humans, verifies compact table commit-history evidence, exports QueryGraph bootstrap artifacts, drains the outbox, and proves the resulting bundle through QueryGraph's Rust verifier/importer. It then asks LakeCat to verify its own compact handoff summary and recompute the raw artifact file hashes, making the summary a first-class acceptance artifact rather than an unchecked convenience file. It is small, but it is not decorative. It is the acceptance story for a catalog that participates in the user workflow from notebook to agent. The summary file gives

automation a single stable place to find the accepted table/view counts, semantic hashes, bundle, lineage drain, import plan, captured verifier outputs, and raw artifact hashes without scraping terminal text.

1.24 Operating The Book's Example System

The local development posture is intentionally small:

```
cargo run -p lakecat-cli -- config
cargo run -p lakecat-cli -- storage-profile-list
cargo run -p lakecat-cli -- policy-list
cargo run -p lakecat-cli --features qglake-fixture -- qglake-fixture \
  --output target/qglake/lakecat-bootstrap.json \
  --drain-output target/qglake/lineage-drain.json \
  --principal did:example:agent
cargo run -p lakecat-cli -- qglake-verify-replay \
  --bundle target/qglake/lakecat-bootstrap.json \
  --drain target/qglake/lineage-drain.json \
  --principal did:example:agent
scripts/qglake-handoff-local.sh
cargo run -p lakecat-cli -- qglake-verify-handoff \
  --summary target/qglake-handoff/handoff-summary.json \
  --json
cargo run -p lakecat-cli -- bootstrap-export \
  --output lakecat-bootstrap.json
```

The important thing is what these commands exercise. They are not a separate product surface. They touch the same catalog, policy, bootstrap, and QueryGraph export contracts that the service uses.

1.25 What Comes Next

LakeCat is a direction more than a single release. The next slices should keep making the architecture more true:

1. Remove temporary Sail patch bridges when the required helpers are available upstream.
2. Keep pushing reusable Iceberg table-status, planning, metadata-as-data, and commit helpers into Sail.
3. Make the transactional outbox the only path for graph and lineage side effects.
4. Expand TypeSec-backed capability checks until every privileged path is unbypassable.
5. Keep graph taxonomy and Cypher behavior in Grust.
6. Keep OSI as a QueryGraph-owned semantic handoff rather than a LakeCat-authored business model.
7. Prove the bootstrap bundle through QueryGraph import on every meaningful public-surface change.

The payoff is a catalog foundation that feels ordinary to Iceberg clients and rich to QueryGraph. Tables remain portable. Policies become enforceable. Graph and lineage become replayable. Sail plans close to the data. QueryGraph gets a trustworthy substrate for the next version of its lakehouse intelligence.